Taller Integrador — Banco Andino

Bases de Datos 2

Integrantes:

David Felipe Bejarano Montes
Samuel Camelo Araque

Fecha:13/09/2026

.

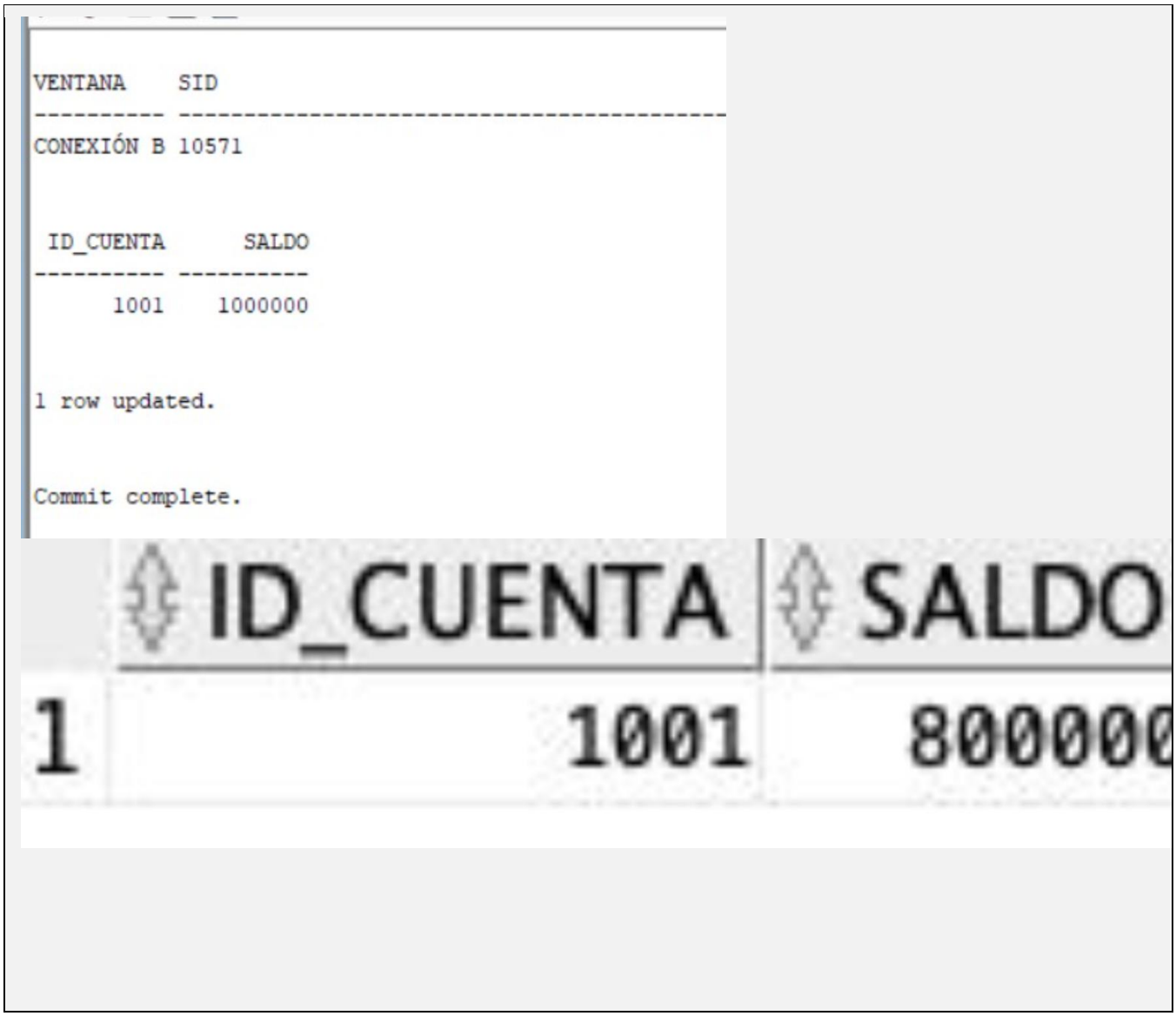
Parte 1 · Transacciones, concurrencia y bloqueos *(15 puntos)*

Archivos: 01_sesion_A.sql y 01_sesion_B.sql

Ejercicio 1.1 — El retiro que se pierde (actualización perdida)

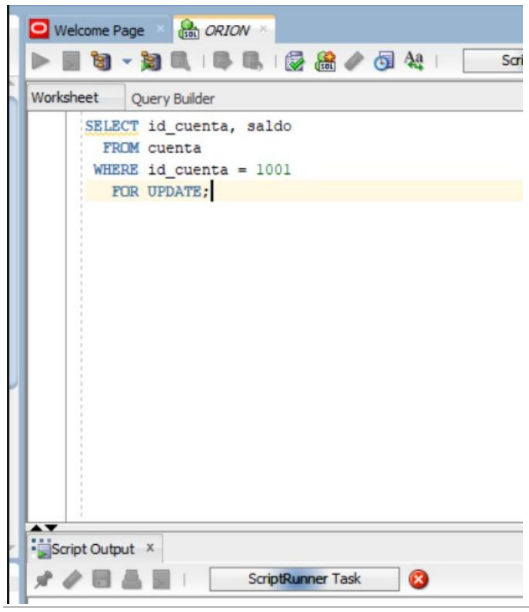
PREGUNTA 1.1 — Salieron 300.000 (A) más 200.000 (B) = 500.000 de 1.000.000. Deberían quedar 500.000. ¿Cuántos quedan? ¿Quién "ganó"? ¿Hubo algún error, alguna advertencia, algún registro de que algo salió mal?

Respuesta: Quedan 800.000, no 500.000. "Ganó" B: su escritura fue la última y sobreescribió por completo el resultado de A. No hay ningún error ni advertencia — Oracle ejecutó dos UPDATEs válidos, uno detrás del otro, exactamente como se le pidió. Desde el punto de vista del motor no pasó nada raro.



PREGUNTA 1.2 — El bloqueo de fila SÍ funcionó: B esperó a que A confirmara. Explique entonces por qué el banco perdió dinero de todos modos.

Respuesta: El bloqueo de fila evitó que las dos escrituras ocurrieran *al mismo tiempo* (evitó la corrupción física), pero no evitó que B calculara su UPDATE con un dato viejo. B escribió un valor absoluto (800000) que ya traía calculado desde antes de que A confirmara. El bloqueo solo pospuso la escritura de B; no la obligó a recalcular sobre el valor fresco**.**



PREGUNTA 1.3 — En un marketplace este error vende una unidad de más. En un banco, ¿de quién es la plata que se acaba de crear de la nada, y quién la va a reclamar?

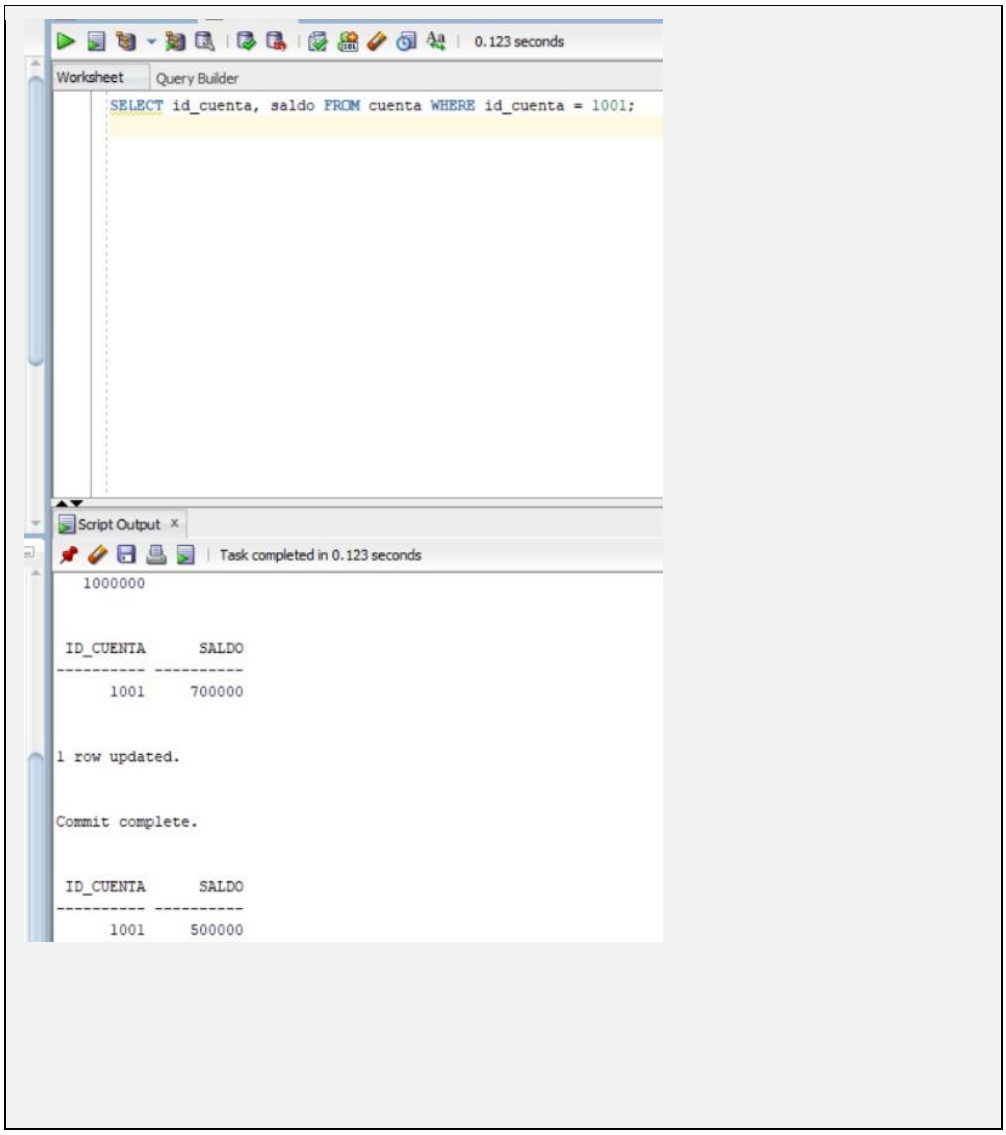
Respuesta: Esos 300.000 no son de nadie: es un hueco contable. El sistema "cree" que hay 300.000 más de los que realmente debería tener esa cuenta según las operaciones que de verdad ocurrieron. Lo va a reclamar el área de conciliación/auditoría cuando cuadre movimientos vs. saldos, y hasta entonces el banco está exhibiendo una cifra que no corresponde a la realidad operativa — es un pasivo oculto para el banco, no una ganancia para el cliente que nadie note a tiempo.

Ejercicio 1.2 — Defensa pesimista: SELECT ... FOR UPDATE

PREGUNTA 1.4 — Ahora sí quedan 500.000. ¿Qué dos cosas cambiaron respecto al ejercicio anterior? (pista: una es el FOR UPDATE, la otra está dentro del UPDATE mismo)

Respuesta: B ya no lee con un SELECT normal sino con FOR UPDATE, así que quien intente tomar la fila con intención de escribir queda bloqueado (no solo el UPDATE final, sino también su lectura).

El UPDATE de B pasó de ser un valor absoluto a uno relativo: UPDATE cuenta SET saldo = saldo - 200000. Cuando el bloqueo se libera, B lee el saldo *actual* de la fila (ya con el descuento de A aplicado) y resta sobre eso, no sobre lo que había leído al principio.



PREGUNTA 1.5 — ¿Cuál fue el precio que se pagó por esta seguridad? Piense en una cuenta empresarial de recaudo que recibe cientos de consignaciones por minuto.

Respuesta: El precio es contención: mientras A tiene la fila reservada, cualquier otra transacción que necesite escribir (o leer con intención de escribir) sobre esa misma cuenta debe esperar en fila. Para una cuenta empresarial de recaudo con cientos de consignaciones por minuto, esto convierte una operación que debería ser paralela en una cola secuencial: el throughput cae a "una transacción a la vez" sobre esa fila, con riesgo de timeouts y de que la cola crezca más rápido de lo que se vacía.

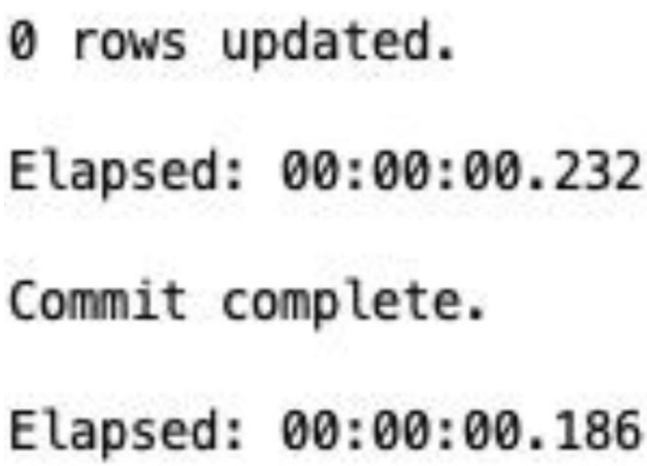
Ejercicio 1.3 — Defensa optimista: columna de versión

PREGUNTA 1.6 — Su UPDATE afectó 0 filas. ¿Qué debe hacer la aplicación en ese momento, y qué NO debe hacer nunca?

Respuesta: B lee versión 0 y escribe primero: su UPDATE ... WHERE version = 0 sí encuentra la fila, la actualiza y la deja en version = 1, y confirma. Cuando A intenta su UPDATE ... WHERE version = 0 (la versión que *él* leyó), esa condición ya no coincide con nada — la fila real está en version = 1 — así que el UPDATE de A afecta 0 filas. No hay excepción ni error SQL; es un no-op silencioso.

Lo que debe hacer la aplicación: revisar el conteo de filas afectadas (SQL%ROWCOUNT o su equivalente en la capa de aplicación) y, si es 0, tratarlo como un conflicto de concurrencia — releer el saldo y la versión actuales, recalcular la operación sobre ese valor fresco, y reintentar el UPDATE con la nueva versión.

Lo que NUNCA debe hacer: asumir que la operación fue exitosa solo porque no hubo error (el motor no avisa por sí solo), ni reintentar ciegamente con la versión vieja (fallaría siempre), ni continuar el flujo de negocio (por ejemplo, entregar el dinero por el cajero) como si el descuento en base de datos sí se hubiera aplicado.



PREGUNTA 1.7 — Compare las dos defensas. ¿En qué producto del banco usaría la pesimista y en cuál la optimista? Justifique con el nivel de contención esperado, no con opiniones.

Respuesta:

Pesimista (FOR UPDATE): úsela donde el choque es *poco frecuente pero costoso si ocurre* — por ejemplo, una transferencia puntual de alto valor entre dos cuentas específicas. El bloqueo es corto y nadie más suele estar tocando esas dos filas al mismo tiempo, así que el costo de esperar es bajo.

Optimista (versión): úsela donde el recurso se **lee** todo el tiempo pero se **escribe** rara vez por procesos distintos — por ejemplo, el saldo de una cuenta personal, consultado constantemente por la app móvil pero modificado solo cuando su dueño hace una operación. Bloquear ahí penalizaría a miles de lectores por un conflicto que casi nunca pasa.

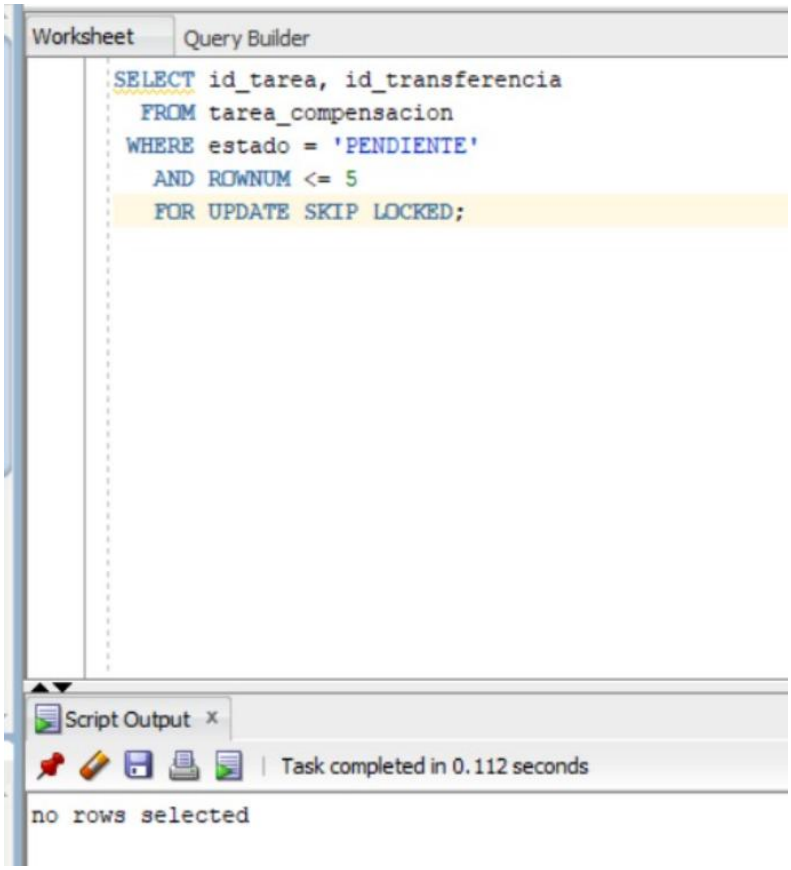
La cuenta empresarial de recaudo del ejercicio 1.5 es justamente el caso que **no** debería usar pesimista: alta frecuencia de escritura concurrente + bloqueo por fila = cola larga. Ahí optimista con reintento es mejor, aunque implique más código en la app.

Ejercicio 1.4 — Repartir la cola de compensación: SKIP LOCKED

PREGUNTA 1.8 — ¿Qué habría pasado sin SKIP LOCKED? Describa los dos comportamientos posibles según se use FOR UPDATE simple o NOWAIT.

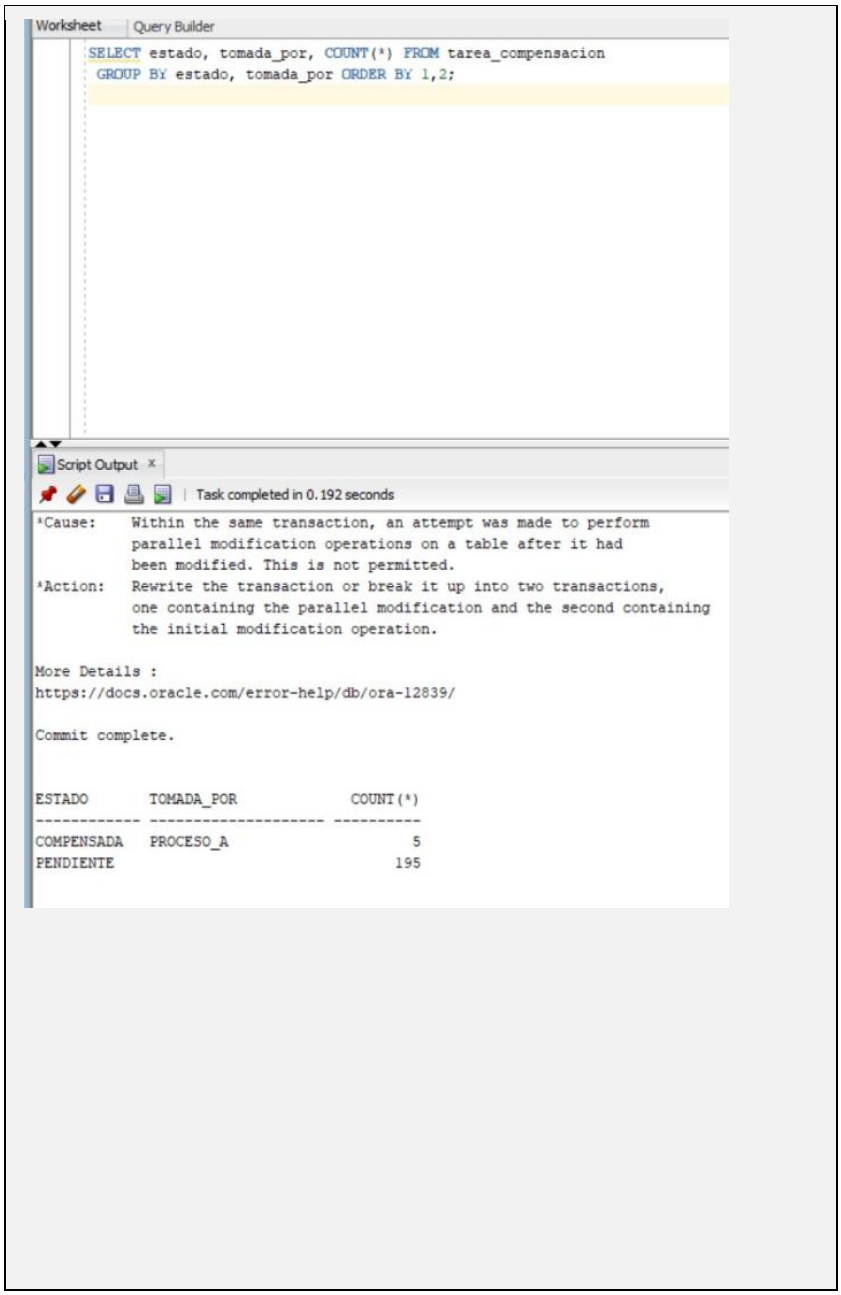
Respuesta: FOR UPDATE simple (sin NOWAIT): B se queda colgado esperando a que A libere las filas que ya tomó, aunque a B no le importaba exactamente cuáles, sino tener 5 cualquiera. Rompe el propósito de tener dos procesos compensando en paralelo.

FOR UPDATE NOWAIT: B recibe de inmediato ORA-00054: resource busy, y la aplicación tendría que atrapar ese error y reintentar la consulta en un bucle manual — funciona, pero es más código y más frágil que dejar que Oracle simplemente le entregue otras filas.



PREGUNTA 1.9 — En una compensación real, procesar dos veces la misma transferencia significa mover la plata dos veces. ¿Qué le garantiza SKIP LOCKED que no le garantizaría un reparto hecho en la aplicación con un "WHERE MOD(id,2) = 0" para un proceso y "= 1" para el otro?

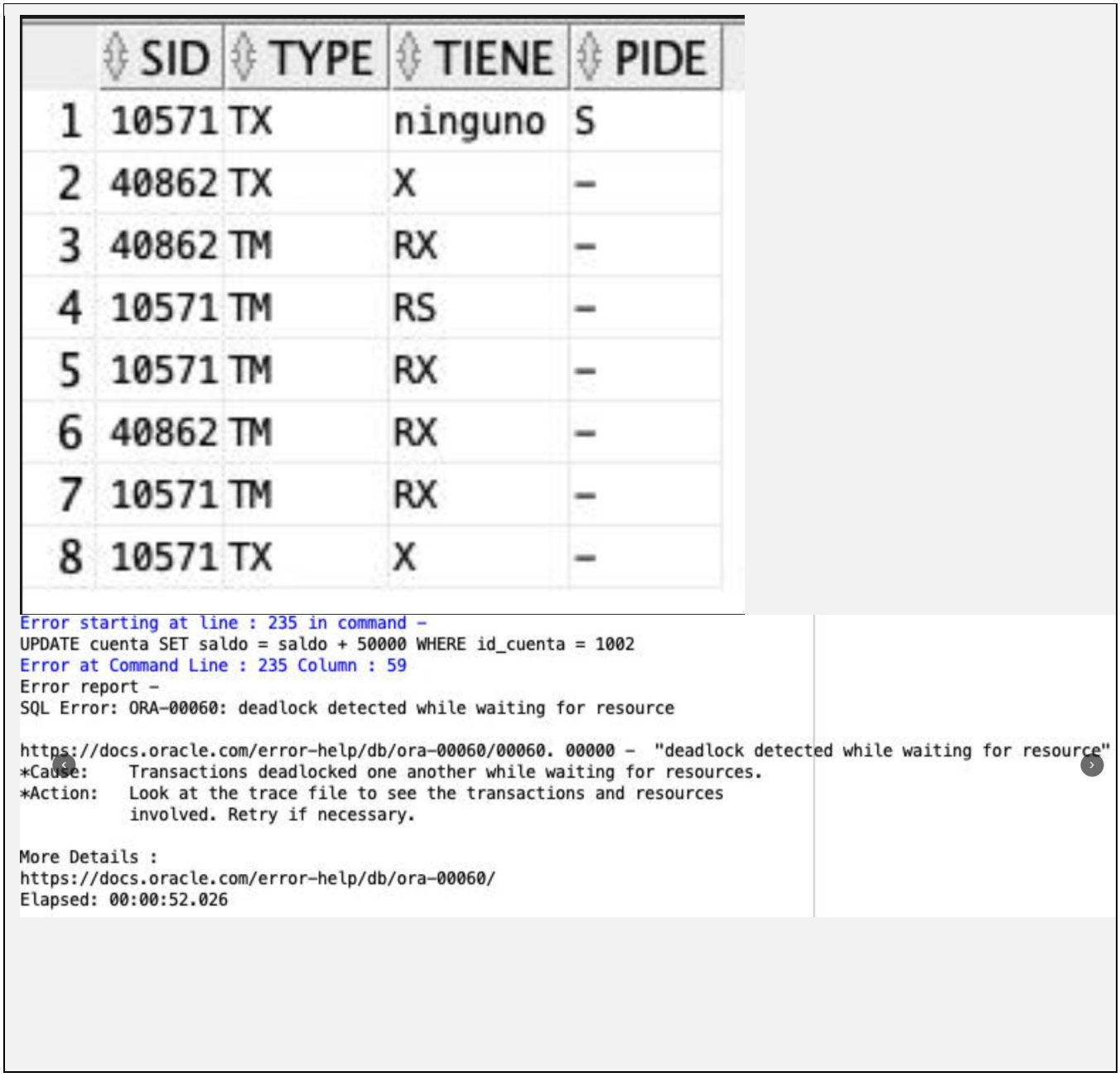
Respuesta: SKIP LOCKED es una garantía del motor, no de la lógica de la aplicación: es *imposible* que dos sesiones terminen con la misma fila, sin importar cuántos procesos haya, en qué orden arranquen, o si uno se cae a mitad de camino (su bloqueo desaparece con el ROLLBACK/fin de sesión y la tarea vuelve a quedar disponible). Un reparto por MOD(id,2) es una convención de la aplicación: si alguien agrega un tercer proceso y nadie actualiza el MOD, si dos instancias del mismo proceso arrancan por error, o si un proceso se cae y otro lo reemplaza sin saber qué tareas ya tomó el caído, nada en la base de datos impide que dos procesos muevan la misma plata dos veces. SKIP LOCKED además se autobalancea: si A es más rápido, toma más tareas; el MOD reparte siempre 50/50 aunque un proceso esté caído.



Ejercicio 1.5 — El bloqueo que nadie espera: FK sin índice

PREGUNTA 1.10 — Explique por qué Oracle toma un bloqueo de TABLA sobre el hijo cuando se borra un padre y la FK no está indexada.

Respuesta: Cuando se borra (o se actualiza la PK de) una fila padre, Oracle tiene que garantizar que ninguna fila hija quede huérfana. Si la columna FK del hijo no está indexada, no hay forma eficiente de verificar "¿alguien más referencia esta fila?": tocaría recorrer toda la tabla hija. Para evitar que, mientras dura esa verificación, alguien inserte justo la fila hija que haría inválido el borrado, Oracle toma un bloqueo de tabla completa (TM) sobre el hijo durante toda la transacción del padre — no un bloqueo de fila, sino de toda la tabla, aunque la fila que se borra no tenga ningún hijo real (como pasó con la transferencia 900005, que no tenía notificaciones).

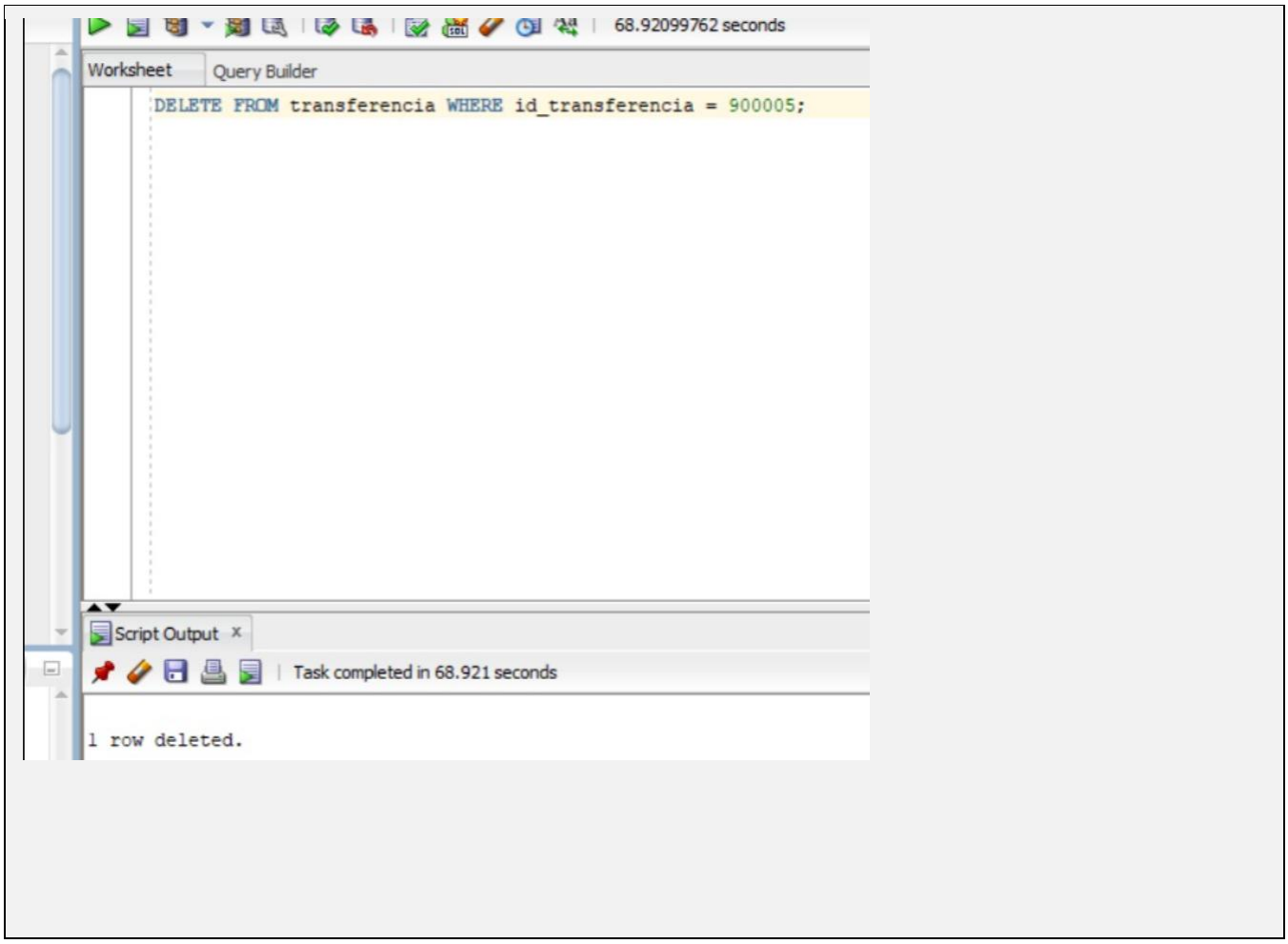


	SID	TYPE	TIENE	PIDE
1	10571	TX	ninguno	S
2	40862	TX	X	-
3	40862	TM	RX	-
4	10571	TM	RS	-
5	10571	TM	RX	-
6	40862	TM	RX	-
7	10571	TM	RX	-
8	10571	TX	X	-

```
Error starting at line : 235 in command -
UPDATE cuenta SET saldo = saldo + 50000 WHERE id_cuenta = 1002
Error at Command Line : 235 Column : 59
Error report -
SQL Error: ORA-00060: deadlock detected while waiting for resource

https://docs.oracle.com/error-help/db/ora-00060/00060. 00000 -  "deadlock detected while waiting for resource"
*Cause:    Transactions deadlocked one another while waiting for resources.
*Action:   Look at the trace file to see the transactions and resources
           involved. Retry if necessary.

More Details :
https://docs.oracle.com/error-help/db/ora-00060/
Elapsed: 00:00:52.026
```



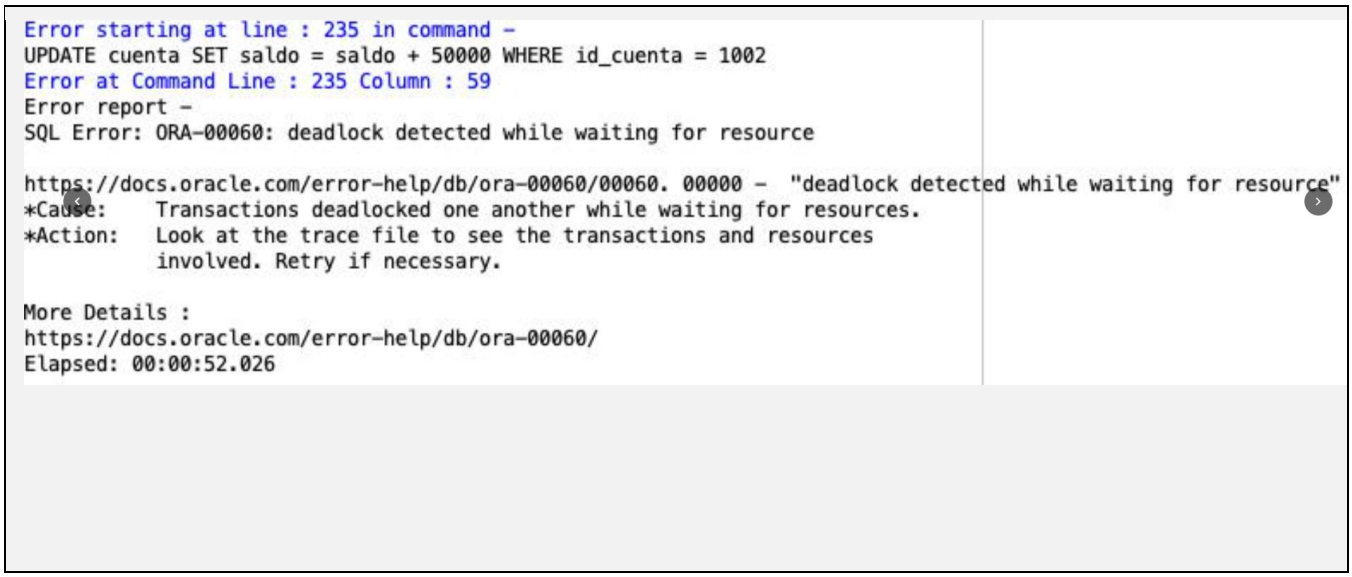
PREGUNTA 1.11 — Revise el esquema completo del Banco Andino. ¿Qué OTRAS llaves foráneas están sin índice? ¿Cuáles de ellas son realmente un problema en este negocio y cuáles no? Justifique pensando en qué se borra y qué no se borra nunca en un banco.

Respuesta: El criterio es: el riesgo depende de si el padre realmente se borra o se actualiza su PK en producción, no de si técnicamente la FK está sin índice. cuenta.id_producto y cuenta.id_sucursal son las dos que de verdad ameritan un índice, porque sus tablas padre (catálogo de productos, sucursales) sí sufren mantenimiento administrativo.

Ejercicio 1.6 — Interbloqueo: las dos transferencias cruzadas

PREGUNTA 1.12 — Oracle abortó UNA sentencia, no la transacción completa. ¿En qué estado queda la sesión víctima y qué debe hacer? ¿Qué pasa si la aplicación asume que Oracle deshizo todo?

Respuesta: Oracle solo revierte la sentencia que causó el ORA-00060 (la última que se quedó esperando y cerró el ciclo), no toda la transacción. La sesión víctima queda con su transacción abierta pero a medias: por ejemplo A ya debitó su origen (paso 1) y esa parte sigue sin confirmar cuando su segundo UPDATE (paso 3) es el que aborta. La aplicación debe hacer ROLLBACK explícito de toda la transacción — si asume que Oracle "ya deshizo todo" y sigue adelante sin ese ROLLBACK, puede terminar confirmando (COMMIT) una transacción a medio hacer: en este caso, una cuenta debitada sin que la otra se haya acreditado.



PREGUNTA 1.13 — Escriba la regla de diseño, en una frase, que habría hecho imposible este interbloqueo. Después dígala en términos de código: ¿qué línea concreta hay que agregar antes de los dos UPDATE?

Respuesta: La regla: bloquear siempre los recursos compartidos en el mismo orden, sin importar el sentido de la operación de negocio — por ejemplo, siempre por id_cuenta ascendente, sin importar cuál es origen y cuál destino.

CODIGO

SELECT id_cuenta FROM cuenta WHERE id_cuenta IN (:cta_origen, :cta_destino) ORDER BY id_cuenta FOR UPDATE;

Si las dos transferencias cruzadas siempre piden primero la cuenta de menor id_cuenta, nunca se forma el ciclo A-espera-a-B-espera-a-A.

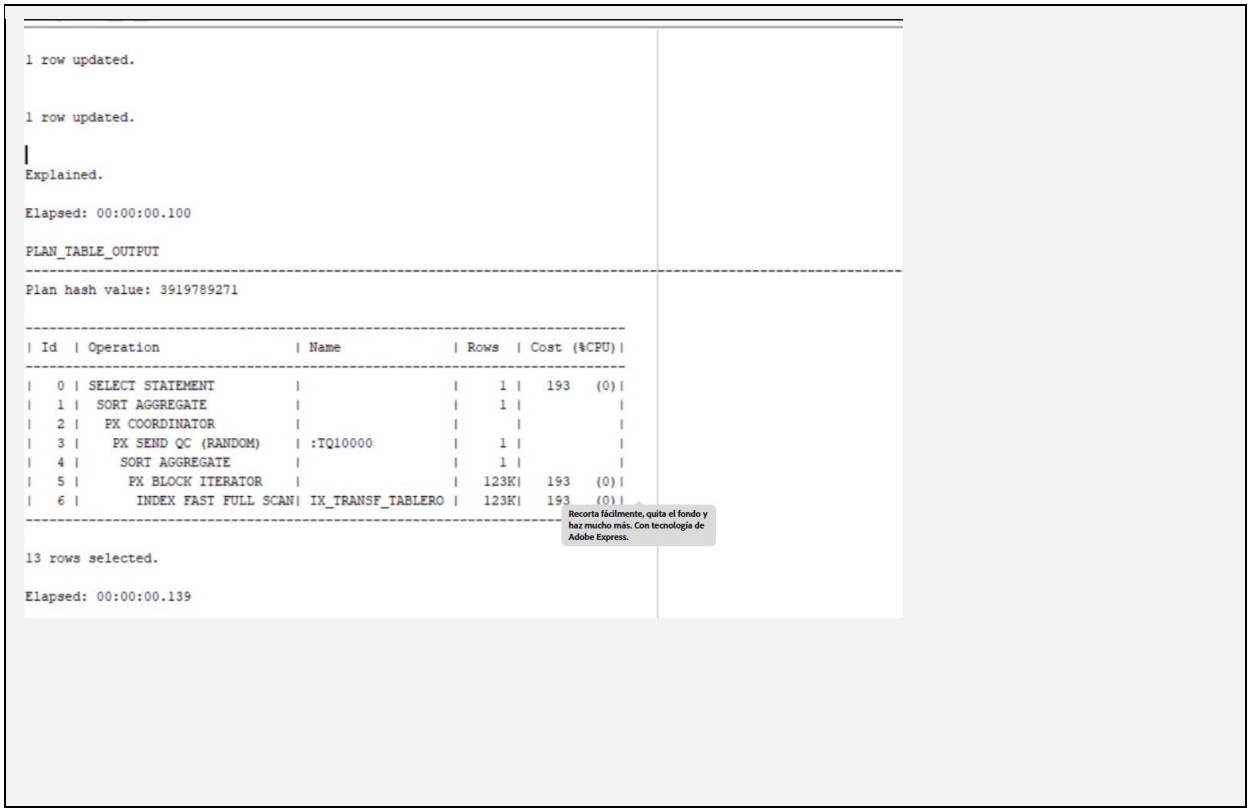
Parte 2 · Índices y optimización de consultas *(15 puntos)*

Archivo: 02_indices.sql — use F10 (Explain Plan), F6 (Autotrace) y F5 (Run Script). Anote siempre COST y consistent gets.

Ejercicio 2.1 — El informe de la Superintendencia

PREGUNTA 2.1 — La tabla tiene 200.000 filas y la consulta devuelve unos pocos miles. ¿Por qué el optimizador está leyendo la tabla completa? ¿Es un error del optimizador?

Respuesta: No es un error del optimizador: es la única opción que tiene. Sin ningún índice sobre fecha_transf ni estado, la única forma de evaluar el filtro es leer cada fila y comprobarla — eso es exactamente lo que hace un FULL TABLE SCAN. El optimizador no "se equivoca", está tomando la decisión correcta *dado lo que existe*; el problema no es de él, es que falta un índice.



```
1 row updated.


1 row updated.

Explained.

Elapsed: 00:00:00.100

PLAN_TABLE_OUTPUT
--------------------------------------------------------------------------------------------------------------------
Plan hash value: 3919789271

-------------------------------------------------------------------------------
| Id  | Operation                | Name              | Rows  | Cost (%CPU)|
-------------------------------------------------------------------------------
|   0 | SELECT STATEMENT         |                   |     1 |   193   (0)|
|   1 |  SORT AGGREGATE          |                   |     1 |            |
|   2 |   PX COORDINATOR         |                   |       |            |
|   3 |    PX SEND QC (RANDOM)   | :TQ10000          |     1 |            |
|   4 |     SORT AGGREGATE       |                   |     1 |            |
|   5 |      PX BLOCK ITERATOR   |                   |   123K|   193   (0)|
|   6 |       INDEX FAST FULL SCAN| IX_TRANSF_TABLERO |   123K|   193   (0)|
-------------------------------------------------------------------------------

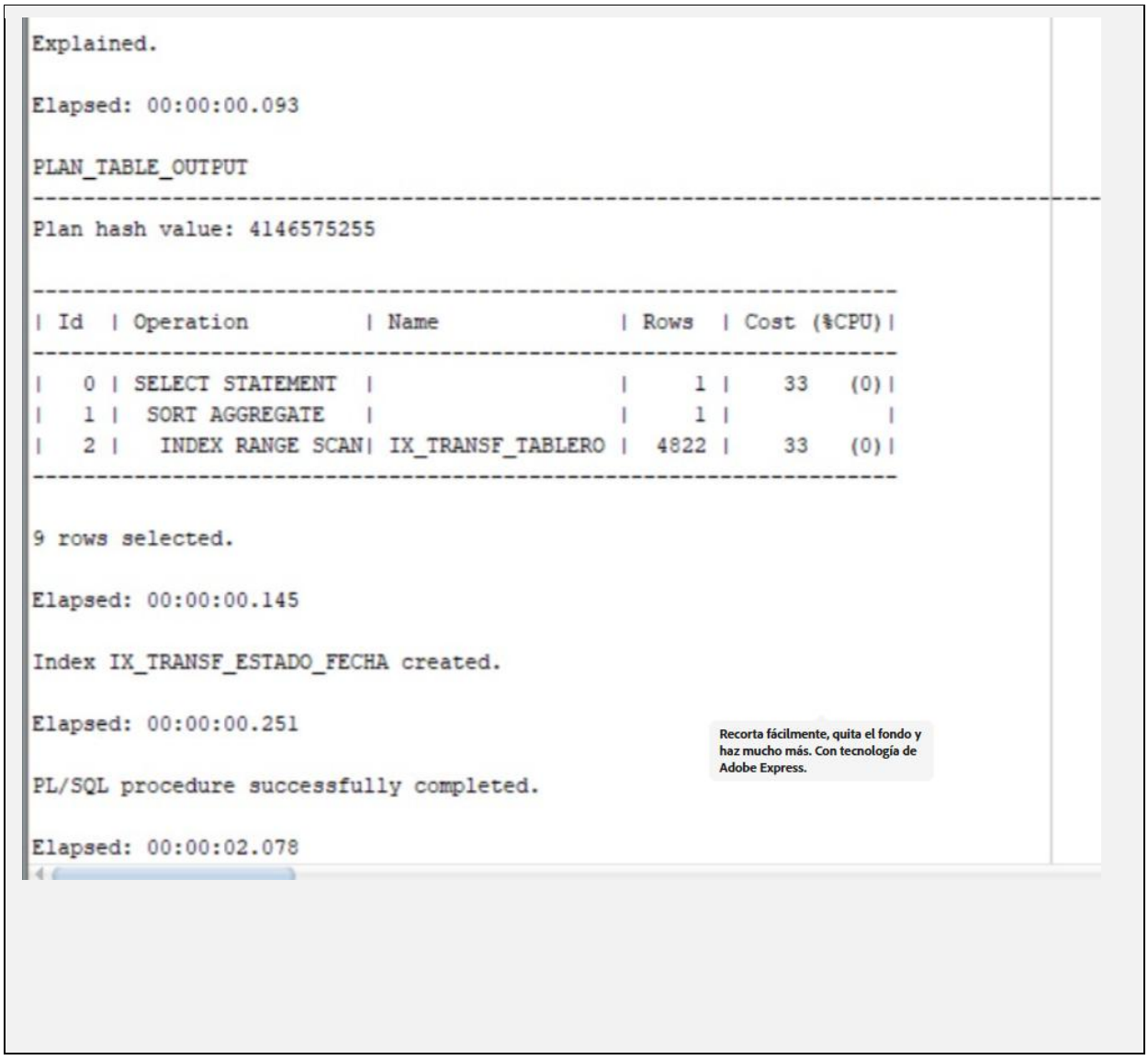
13 rows selected.

Elapsed: 00:00:00.139
```

Recorta fácilmente, quita el fondo y haz mucho más. Con tecnología de Adobe Express.

PREGUNTA 2.2 — ¿Cuánto bajó el COST? Ejecute también con F6 (Autotrace) antes y después y compare los "consistent gets".

Respuesta: Debería bajar bastante: el plan pasa de TABLE ACCESS FULL a INDEX RANGE SCAN sobre ix_transf_fecha_estado (Oracle puede usar ambas columnas como *access predicate*, o sea como llave de inicio/fin del recorrido) seguido de TABLE ACCESS BY INDEX ROWID — porque SUM(valor) necesita una columna que no está en el índice, así que igual toca visitar la tabla, pero solo para las filas que ya sabe que cumplen, no para las 200.000.



```
Explained.

Elapsed: 00:00:00.093

PLAN_TABLE_OUTPUT
--------------------------------------------------------------------------------
Plan hash value: 4146575255

-----------------------------------------------------------------------
| Id  | Operation         | Name               | Rows  | Cost (%CPU)|
-----------------------------------------------------------------------
|   0 | SELECT STATEMENT  |                    |     1 |    33   (0)|
|   1 |  SORT AGGREGATE   |                    |     1 |            |
|   2 |   INDEX RANGE SCAN| IX_TRANSF_TABLERO  |  4822 |    33   (0)|
-----------------------------------------------------------------------

9 rows selected.

Elapsed: 00:00:00.145

Index IX_TRANSF_ESTADO_FECHA created.

Elapsed: 00:00:00.251

PL/SQL procedure successfully completed.

Elapsed: 00:00:02.078
```

Recorta fácilmente, quita el fondo y haz mucho más. Con tecnología de Adobe Express.

PREGUNTA 2.3 — ¿Cuál de los dos índices elige el optimizador cuando no se le fuerza nada? Compare el COST de los dos SELECT anteriores. Formule la regla general sobre el orden de columnas en un índice compuesto, distinguiendo el predicado de IGUALDAD del de RANGO.

Respuesta: Regla general: en un índice compuesto, las columnas con predicado de igualdad deben ir primero, y las de rango al final. Con (estado, fecha_transf) (igualdad primero), Oracle puede construir una llave de inicio/fin usando las dos columnas — arranca directo en ('APLICADA', '2026-03-01') y para en ('APLICADA', '2026-04-01'), sin tocar ninguna entrada de otro estado. Con (fecha_transf, estado) (rango primero), Oracle solo puede usar fecha_transf para la llave de inicio/fin; estado queda como filtro *dentro* del rango, así que recorre todas las entradas de ese mes sin importar el estado, y luego descarta las que no sirven. Por eso, si se lo dejas sin forzar, el optimizador normalmente prefiere ix_transf_estado_fecha para *esta consulta específica* — menos entradas de índice recorridas.

```
Explained.

Elapsed: 00:00:00.098

PLAN_TABLE_OUTPUT
--------------------------------------------------------------------------
Plan hash value: 2789700739

-----------------------------------------------------
| Id  | Operation            | Name     | Cost (%CPU)|
-----------------------------------------------------
|   0 | SELECT STATEMENT     |          |    27   (0)|
|   1 |  PX COORDINATOR      |          |            |
|   2 |   PX SEND QC (RANDOM)| :TQ10000 |    27   (0)|
|   3 |    PX BLOCK ITERATOR |          |    27   (0)|
|   4 |     TABLE ACCESS FULL| CLIENTE  |    27   (0)|
-----------------------------------------------------

11 rows selected.

Elapsed: 00:00:00.122

Index IX_CLIENTE_NOMBRE created.

Elapsed: 00:00:00.150

PL/SQL procedure successfully completed.
```

Recorta fácilmente, quita el fondo y haz mucho más. Con tecnología de Adobe Express.

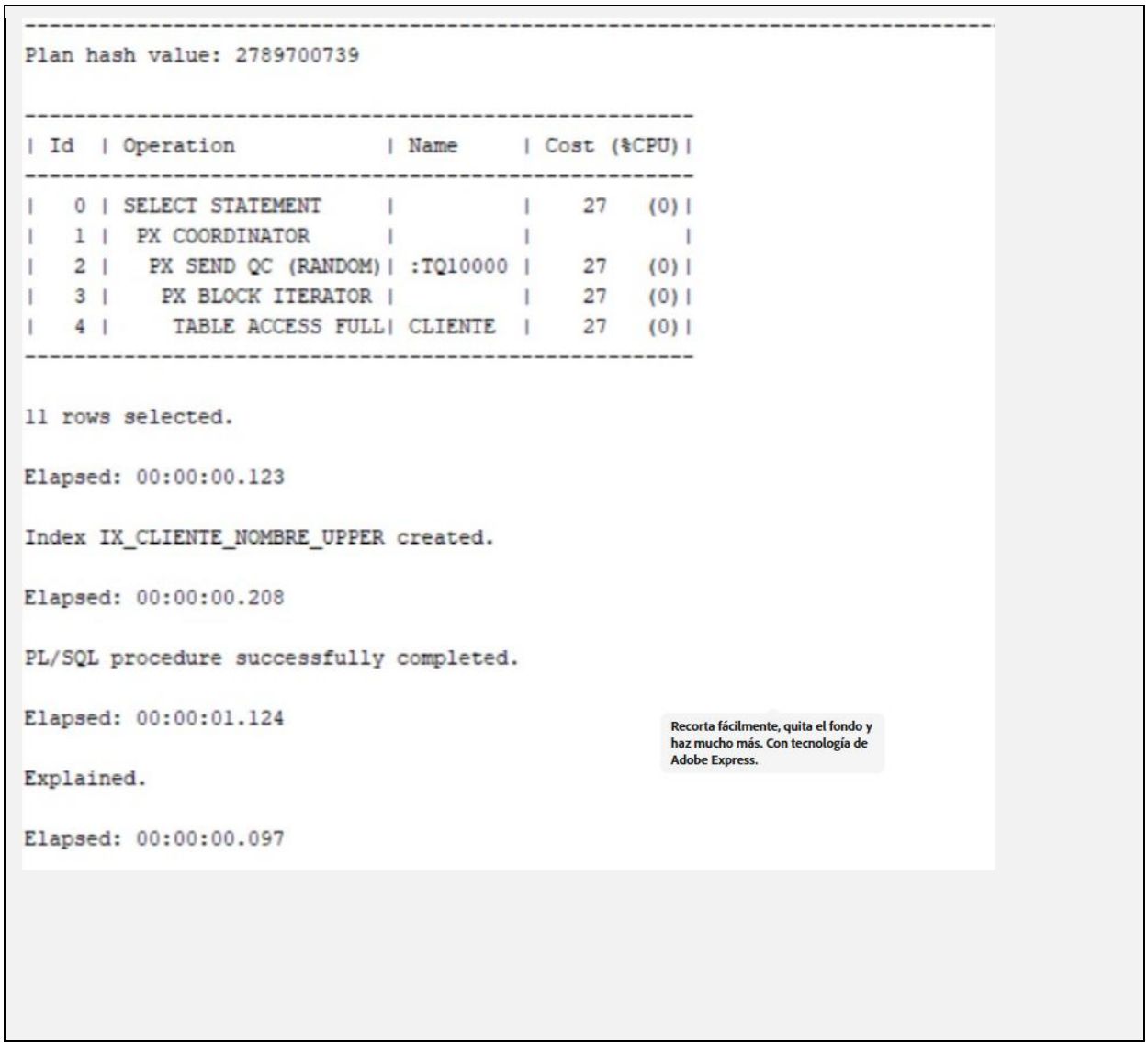
PREGUNTA 2.4 — Suponga ahora que existe además un informe diario que filtra SOLO por rango de fechas, sin estado. ¿Cuál de los dos índices le sirve a ese informe y cuál no? ¿Cambia eso su recomendación? Responda cuál de los dos conservaría en producción y por qué.

Respuesta: El informe diario que filtra SOLO por fecha (sin estado) puede seguir usando ix_transf_fecha_estado (fecha como columna líder, sirve para un rango puro), pero no puede usar ix_transf_estado_fecha en absoluto — un índice con estado como columna líder es inútil si la consulta no trae ningún filtro sobre estado. Por eso, aunque ix_transf_estado_fecha gane en costo puntual para la consulta 2.1, la recomendación de producción es quedarse con ix_transf_fecha_estado: sirve a dos patrones de consulta distintos, no solo a uno. Es justo lo que el script hace al final del ejercicio (DROP INDEX ix_transf_estado_fecha).

Ejercicio 2.2 — El índice que existe y no se usa

PREGUNTA 2.5 — El índice existe y sigue haciendo FULL SCAN. ¿Por qué? ¿Qué le hizo la consulta a la columna indexada?

Respuesta: La consulta filtra por UPPER(nombre), no por nombre. Un índice normal (ix_cliente_nombre) guarda los valores tal cual están en la columna, sin transformar; como el predicado le aplica una función a la columna, ese índice no le sirve a Oracle para resolverlo — necesita un índice basado en función (ix_cliente_nombre_upper ON cliente(UPPER(nombre))), que sí guarda el valor ya transformado y por eso sí calza con el predicado.



```
-------------------------------------------------------------------------------
Plan hash value: 2789700739

-----------------------------------------------------
| Id  | Operation            | Name     | Cost (%CPU)|
-----------------------------------------------------
|   0 | SELECT STATEMENT     |          |    27   (0)|
|   1 |  PX COORDINATOR      |          |            |
|   2 |   PX SEND QC (RANDOM)| :TQ10000 |    27   (0)|
|   3 |    PX BLOCK ITERATOR |          |    27   (0)|
|   4 |     TABLE ACCESS FULL| CLIENTE  |    27   (0)|
-----------------------------------------------------

11 rows selected.

Elapsed: 00:00:00.123

Index IX_CLIENTE_NOMBRE_UPPER created.

Elapsed: 00:00:00.208

PL/SQL procedure successfully completed.

Elapsed: 00:00:01.124

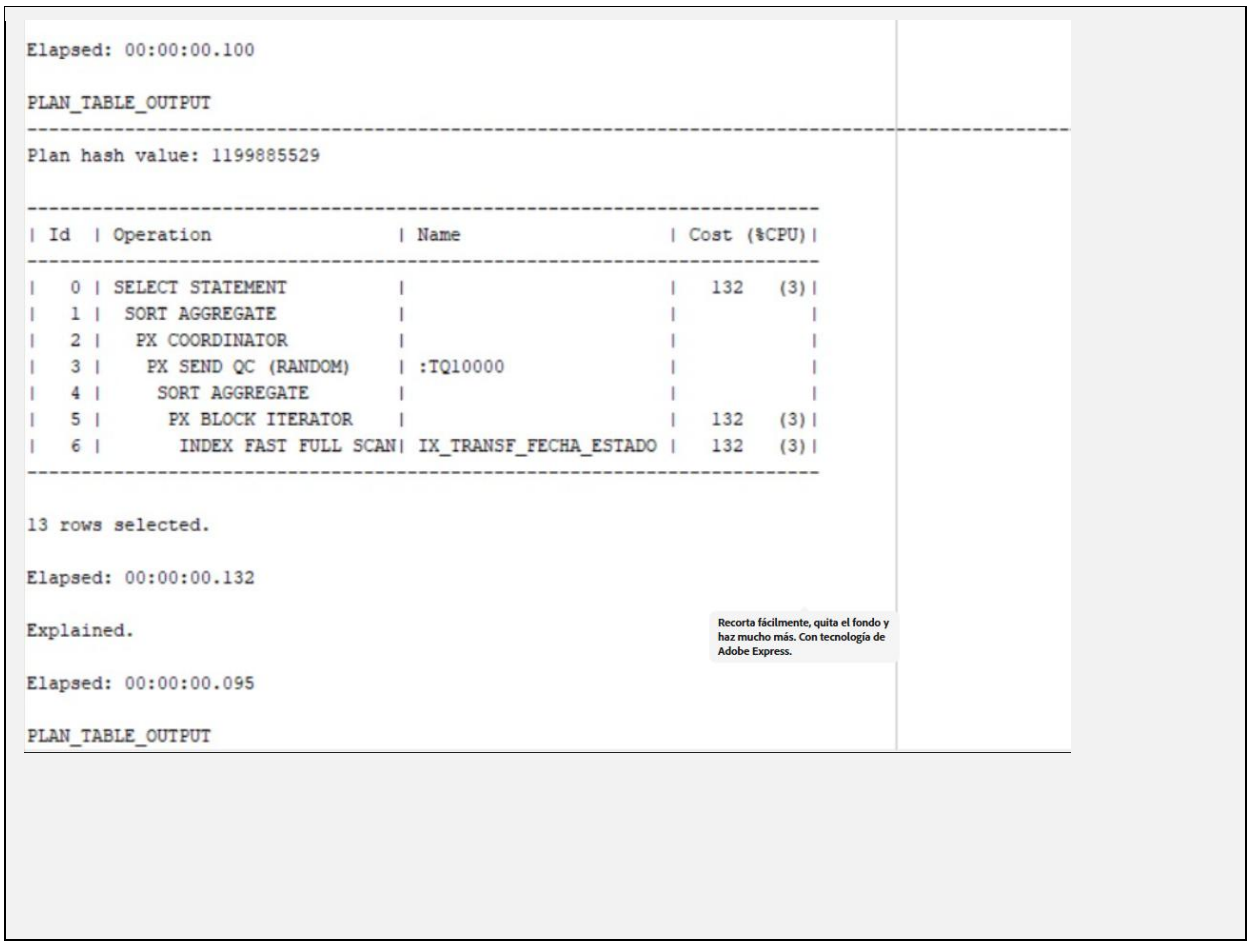
Explained.

Elapsed: 00:00:00.097
```

Recorta fácilmente, quita el fondo y haz mucho más. Con tecnología de Adobe Express.

PREGUNTA 2.6 — Las dos consultas devuelven exactamente lo mismo. Compare los planes. Escriba la regla, en una frase, que todo desarrollador debería tener presente al filtrar por fecha.

Respuesta: Mismo problema con fechas: TRUNC(fecha_transf) = DATE '2026-03-15' envuelve la columna en una función, así que ningún índice sobre fecha_transf "pelada" sirve — full scan otra vez. La versión reescrita como rango (fecha_transf >= ... AND fecha_transf < ...) no toca la columna con ninguna función, así que sí puede usar ix_transf_fecha_estado. La regla en una frase: nunca envuelva en una función la columna que quiere que use el índice — reescriba la condición como un rango o cree un índice basado en esa función.



```
Elapsed: 00:00:00.100

PLAN_TABLE_OUTPUT
------------------------------------------------------------------------------------------------
Plan hash value: 1199885529

-------------------------------------------------------------------------
| Id  | Operation                | Name                  | Cost (%CPU)|
-------------------------------------------------------------------------
|   0 | SELECT STATEMENT         |                       |   132   (3)|
|   1 |  SORT AGGREGATE          |                       |            |
|   2 |   PX COORDINATOR         |                       |            |
|   3 |    PX SEND QC (RANDOM)   | :TQ10000              |            |
|   4 |     SORT AGGREGATE       |                       |            |
|   5 |      PX BLOCK ITERATOR   |                       |   132   (3)|
|   6 |       INDEX FAST FULL SCAN| IX_TRANSF_FECHA_ESTADO |   132   (3)|
-------------------------------------------------------------------------

13 rows selected.

Elapsed: 00:00:00.132

Explained.

Elapsed: 00:00:00.095

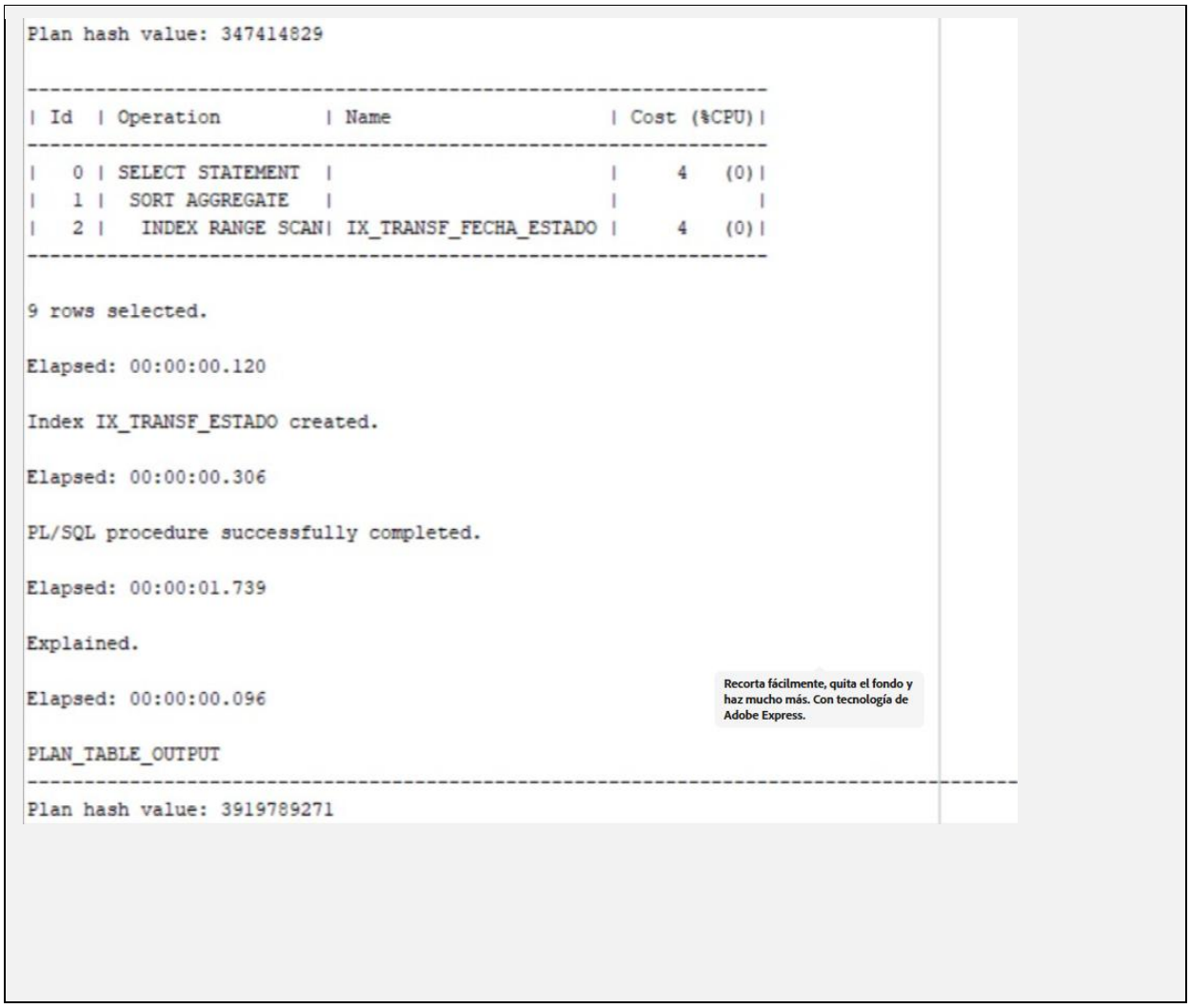
PLAN_TABLE_OUTPUT
```

Recorta fácilmente, quita el fondo y haz mucho más. Con tecnología de Adobe Express.

Ejercicio 2.3 — Cuándo NO conviene un índice

PREGUNTA 2.7 — El mismo índice, la misma consulta, distinto valor: en un caso se usa y en el otro no. ¿Cómo lo sabe el optimizador? ¿Qué información necesita para tomar esa decisión?

Respuesta: El optimizador decide con base en la selectividad estimada de ese valor específico, que saca de las estadísticas (histogramas) sobre la columna estado. Si 'APLICADA' representa una fracción grande de las 200.000 filas (el enunciado menciona ~62%), leer el índice y después ir fila por fila a la tabla (acceso aleatorio) termina costando MÁS que simplemente leer la tabla completa de corrido. Si 'REVERSADA' es un valor raro, el número de filas a traer es pequeño, así que el índice sí gana. Para tomar esta decisión bien, el optimizador necesita estadísticas que reflejen que la columna está desbalanceada (no todos los valores tienen la misma frecuencia) — idealmente un histograma, no solo el conteo de valores distintos.



```
Plan hash value: 347414829

-----------------------------------------------------------------------
| Id  | Operation         | Name                   | Cost (%CPU)|
-----------------------------------------------------------------------
|   0 | SELECT STATEMENT  |                        |     4   (0)|
|   1 |  SORT AGGREGATE   |                        |            |
|   2 |   INDEX RANGE SCAN| IX_TRANSF_FECHA_ESTADO |     4   (0)|
-----------------------------------------------------------------------

9 rows selected.

Elapsed: 00:00:00.120

Index IX_TRANSF_ESTADO created.

Elapsed: 00:00:00.306

PL/SQL procedure successfully completed.

Elapsed: 00:00:01.739

Explained.

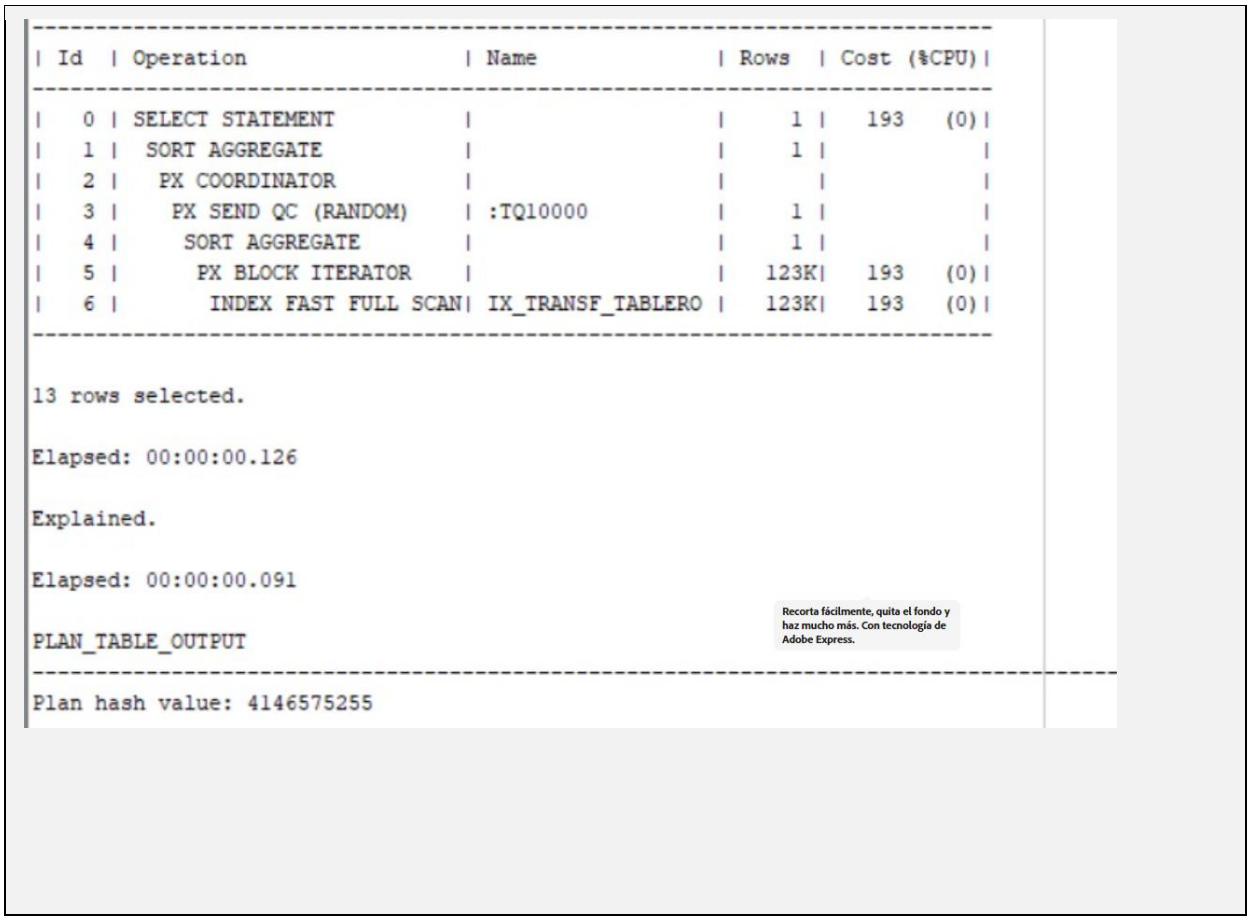
Elapsed: 00:00:00.096

PLAN_TABLE_OUTPUT
-------------------------------------------------------------------------------------
Plan hash value: 3919789271
```

Recorta fácilmente, quita el fondo y haz mucho más. Con tecnología de Adobe Express.

PREGUNTA 2.8 — ¿Qué le dice el CLUSTERING FACTOR de IX_TRANSF_FECHA_ESTADO comparado con el de IX_TRANSF_ESTADO? ¿Por qué son tan distintos?

Respuesta Sobre clustering factor — compara ix_transf_estado contra ix_transf_fecha_estado: si el clustering factor de ix_transf_estado está cerca del número de FILAS de la tabla, significa que filas con el mismo estado están dispersas por bloques físicos distintos (porque el estado de una transferencia no tiene nada que ver con el orden en que se insertó). En cambio, si el clustering factor de ix_transf_fecha_estado está cerca del número de BLOQUES (mucho más bajo), es porque las transferencias probablemente se insertaron en orden cronológico, así que filas con fechas cercanas también quedan físicamente cercanas en la tabla — cada paso del índice cae casi siempre en el mismo bloque o en uno vecino, en vez de saltar por toda la tabla.



```
-------------------------------------------------------------------------------
| Id  | Operation                | Name               | Rows  | Cost (%CPU)|
-------------------------------------------------------------------------------
|   0 | SELECT STATEMENT         |                    |     1 |   193   (0)|
|   1 |  SORT AGGREGATE          |                    |     1 |            |
|   2 |   PX COORDINATOR         |                    |       |            |
|   3 |    PX SEND QC (RANDOM)   | :TQ10000           |     1 |            |
|   4 |     SORT AGGREGATE       |                    |     1 |            |
|   5 |      PX BLOCK ITERATOR   |                    |  123K|   193   (0)|
|   6 |       INDEX FAST FULL SCAN| IX_TRANSF_TABLERO |  123K|   193   (0)|
-------------------------------------------------------------------------------

13 rows selected.

Elapsed: 00:00:00.126

Explained.

Elapsed: 00:00:00.091

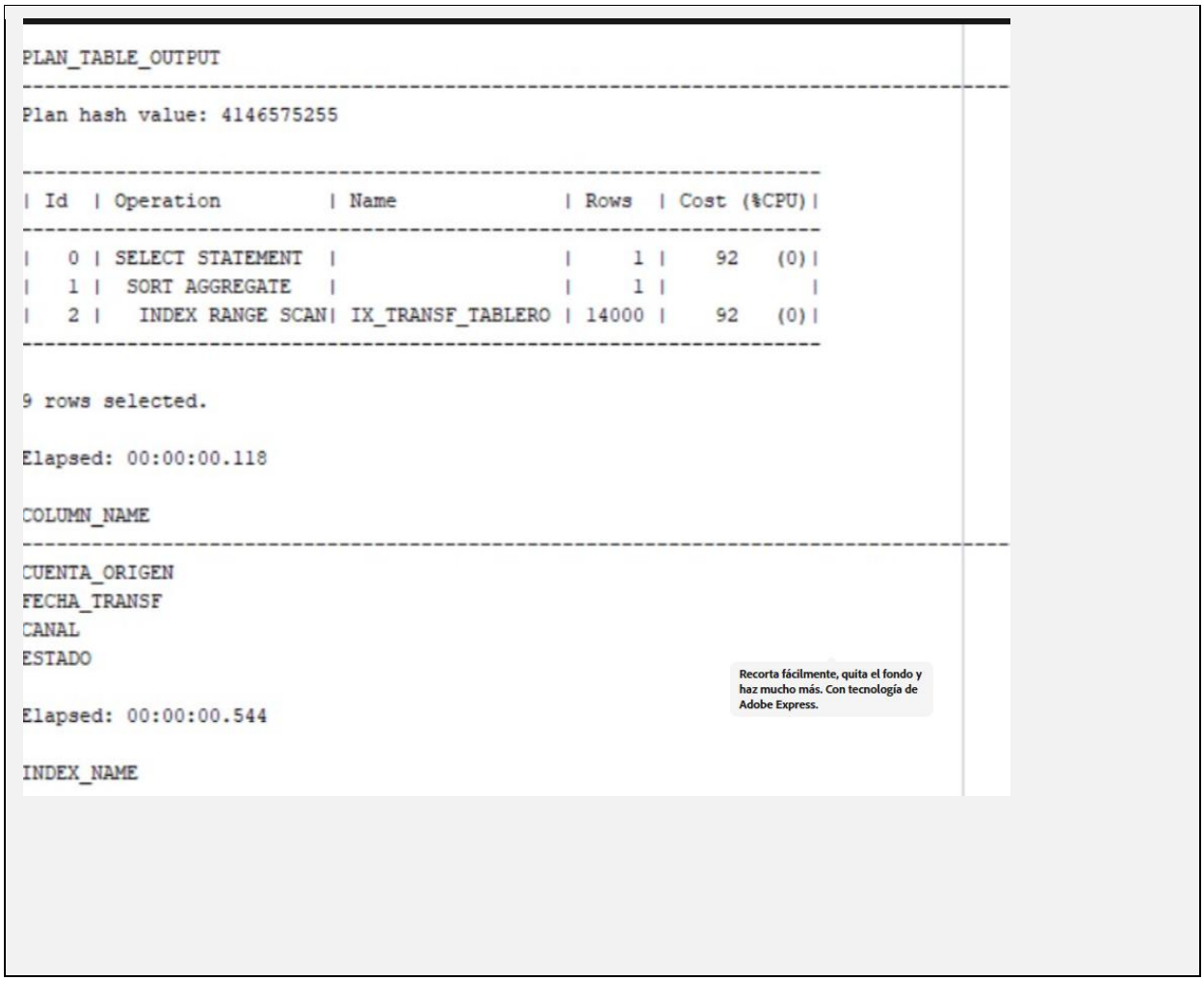
PLAN_TABLE_OUTPUT
-------------------------------------------------------------------------------------------
Plan hash value: 4146575255
```

Recorta fácilmente, quita el fondo y haz mucho más. Con tecnología de Adobe Express.

Ejercicio 2.4 — El índice que evita tocar la tabla

PREGUNTA 2.9 — Desapareció el TABLE ACCESS. ¿Cómo se llama esta técnica y por qué es tan efectiva? ¿Cuál es su límite: hasta dónde tiene sentido seguir agregando columnas al índice?

Respuesta: Se llama índice de cobertura (*covering index*) o acceso "solo por índice" (*index-only access*). Es tan efectivo porque el paso más caro de un plan basado en índice normalmente es TABLE ACCESS BY INDEX ROWID — accesos aleatorios, uno por fila, al heap de la tabla. Si todas las columnas que la consulta necesita (fecha_transf para filtrar, cuenta_origen para agrupar) ya están en las hojas del índice, Oracle nunca necesita ir a la tabla: se ahorra por completo esos accesos aleatorios. El límite: no se puede convertir cualquier índice en "de cobertura" agregándole columnas sin costo — cada columna extra hace el índice más pesado (más bloques hoja, más espacio, y sobre todo más lento cada INSERT/UPDATE/DELETE, porque hay que mantener también esa columna en el índice). La regla práctica es agregar solo las columnas que de verdad se reutilizan en consultas frecuentes, no "por si acaso" — que es justo el gancho hacia la 2.10.



```
PLAN_TABLE_OUTPUT
--------------------------------------------------------------------------------
Plan hash value: 4146575255

-------------------------------------------------------------------------
| Id  | Operation         | Name              | Rows  | Cost (%CPU)|
-------------------------------------------------------------------------
|   0 | SELECT STATEMENT  |                   |     1 |    92   (0)|
|   1 |  SORT AGGREGATE   |                   |     1 |            |
|   2 |   INDEX RANGE SCAN| IX_TRANSF_TABLERO | 14000 |    92   (0)|
-------------------------------------------------------------------------

9 rows selected.

Elapsed: 00:00:00.118

COLUMN_NAME
--------------------------------------------------------------------------------
CUENTA_ORIGEN
FECHA_TRANSF
CANAL
ESTADO

Elapsed: 00:00:00.544

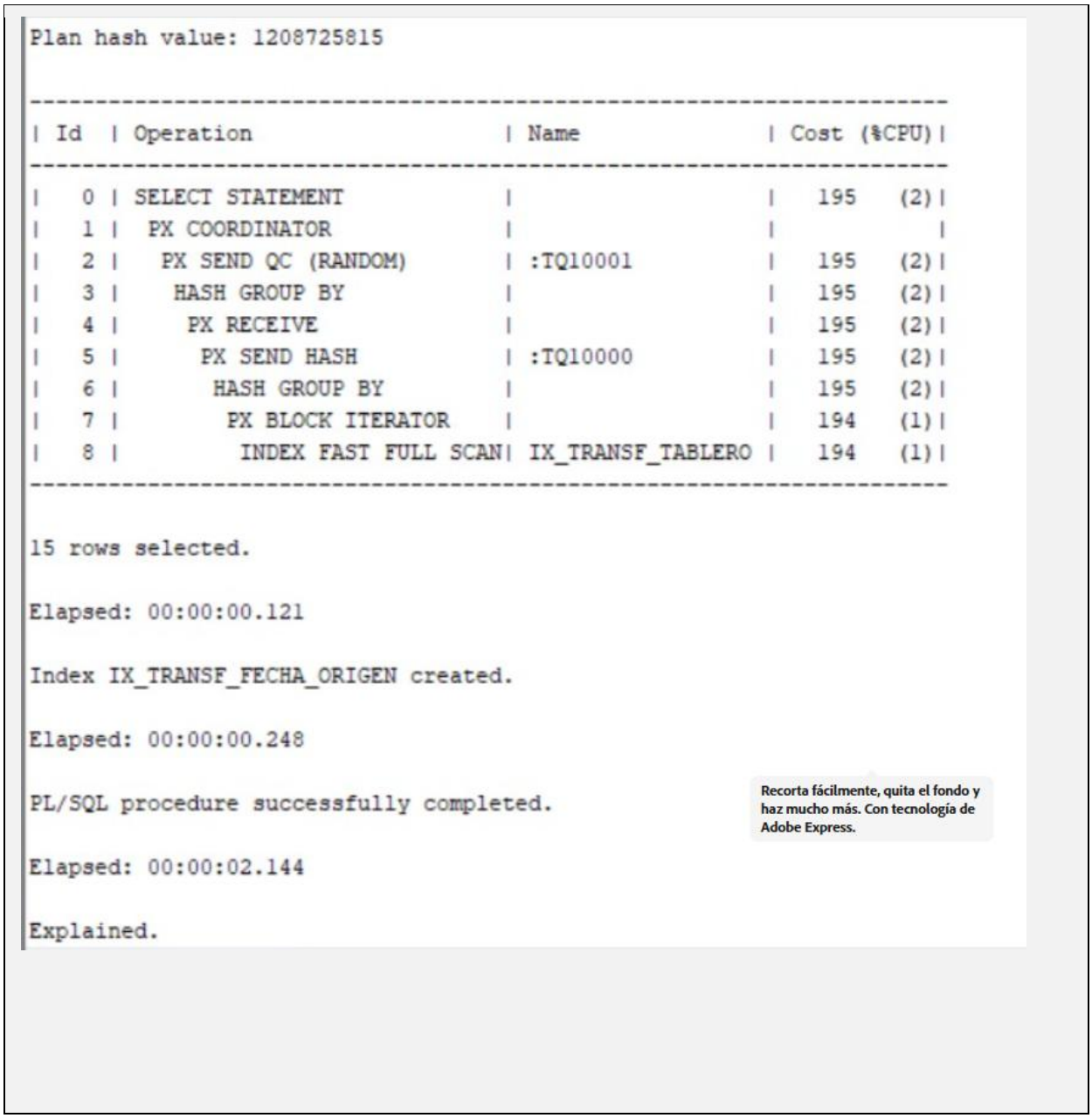
INDEX_NAME
```

Recorta fácilmente, quita el fondo y haz mucho más. Con tecnología de Adobe Express.

Ejercicio 2.5 — Lo que cuesta un índice

PREGUNTA 2.10 — ¿Cuál fue la diferencia de tiempo? Extrapole: el Banco Andino procesa 12.000 transferencias por hora en hora pico. ¿Qué costo operativo tiene cada índice que se agrega "por si acaso"?

Respuesta: Con índices secundarios, cada INSERT no solo escribe en la tabla: también tiene que insertar una entrada en CADA índice secundario (mantenimiento del árbol B, posibles splits de bloques hoja). Sin ellos, el INSERT solo toca la tabla (y la PK). Extrapolando: si el Banco Andino procesa 12.000 transferencias/hora en pico, cada índice "por si acaso" agrega una fracción de tiempo a cada una de esas 12.000 inserciones, todo el tiempo, incluso si la consulta que justificaba ese índice casi nunca se ejecuta — es un costo permanente en escritura a cambio de un beneficio ocasional en lectura.



```
Plan hash value: 1208725815

-----------------------------------------------------------------------
| Id  | Operation                    | Name              | Cost (%CPU)|
-----------------------------------------------------------------------
|   0 | SELECT STATEMENT             |                   |   195   (2)|
|   1 |  PX COORDINATOR              |                   |            |
|   2 |   PX SEND QC (RANDOM)        | :TQ10001          |   195   (2)|
|   3 |    HASH GROUP BY             |                   |   195   (2)|
|   4 |     PX RECEIVE               |                   |   195   (2)|
|   5 |      PX SEND HASH            | :TQ10000          |   195   (2)|
|   6 |       HASH GROUP BY          |                   |   195   (2)|
|   7 |        PX BLOCK ITERATOR     |                   |   194   (1)|
|   8 |         INDEX FAST FULL SCAN| IX_TRANSF_TABLERO |   194   (1)|
-----------------------------------------------------------------------

15 rows selected.

Elapsed: 00:00:00.121

Index IX_TRANSF_FECHA_ORIGEN created.

Elapsed: 00:00:00.248

PL/SQL procedure successfully completed.

Elapsed: 00:00:02.144

Explained.
```

Recorta fácilmente, quita el fondo y haz mucho más. Con tecnología de Adobe Express.

PREGUNTA 2.11 — Proponga la lista DEFINITIVA de índices para este esquema. Para cada uno escriba: qué consulta lo justifica, y por qué no se puede resolver con otro índice ya existente. Tenga en cuenta que TRANSFERENCIA tiene DOS llaves foráneas hacia CUENTA.

Respuesta: Con lo que vieron en todo el ejercicio, la lista defendible es dos que no dejarían en producción: ix_transf_estado (solo) — la 2.7 ya mostró que para el valor dominante el optimizador ni lo usa, así que solo sirve para consultas por un estado raro, un caso de uso débil para justificar el costo de escritura — y ix_transf_estado_fecha, redundante con el de fecha_estado para los patrones que vieron.

Vale la pena que menciones explícitamente que TRANSFERENCIA tiene dos FK hacia CUENTA (cuenta_origen y cuenta_destino) y que un solo índice compuesto no sirve para ambos sentidos — si en su negocio también consultan "cuentas que más *reciben*" (no solo las que más *envían*, que es lo que cubre ix_transf_fecha_origen), necesitarían un índice espejo sobre cuenta_destino.

```
Elapsed: 00:00:00.133

INDEX_NAME                                                
```

```
50,000 rows inserted.

Elapsed: 00:00:01.438

Commit complete.

Elapsed: 00:00:00.089

100,000 rows deleted.

Elapsed: 00:00:03.572

Commit complete.

Elapsed: 00:00:00.092

Index IX_TRANSF_FECHA_ESTADO created.

Elapsed: 00:00:00.233

Index IX_TRANSF_FECHA_ORIGEN created.

Elapsed: 00:00:00.237

PL/SQL procedure successfully completed.

Elapsed: 00:00:02.502
```

Recorta fácilmente, quita el fondo y haz mucho más. Con tecnología de Adobe Express.

Parte 3 · SQL analítico, jerárquico y MERGE *(10 puntos)*

Ejercicio 3.1 — Ranking dentro de grupos

PREGUNTA 3.1 — Cambie DENSE_RANK por RANK y luego por ROW_NUMBER. Explique en qué se diferencian los tres cuando hay empates, y cuál usaría usted para un "top 3" que debe devolver exactamente 3 filas.

Respuesta: ANK, DENSE_RANK y ROW_NUMBER permiten asignar posiciones dentro de un grupo, pero presentan diferencias cuando existen empates. RANK asigna la misma posición a los valores empatados y deja saltos en la numeración posterior. DENSE_RANK también asigna la misma posición a los empates, pero continúa la numeración sin dejar saltos. Por su parte, ROW_NUMBER asigna un número consecutivo y único a cada fila, incluso cuando los valores son iguales. Para un Top 3 que deba devolver exactamente tres registros por grupo, siempre que existan al menos tres registros, utilizaría ROW_NUMBER, ya que garantiza una numeración individual de las filas.

Lo importante es que:

RATIO_TO_REPORT(saldo_captado)

OVER (PARTITION BY id_familia)

se calcula antes de:

WHERE puesto <= 3

Por eso el porcentaje se calcula sobre todos los productos de la familia, no únicamente sobre los tres primeros.

ESCRIBA 3.1 — Consulta: top 3 por familia + % sobre el saldo total de la familia

	FAMILIA	PRODUCTO	CUENTAS	SALDO_CAPTADO	PUESTO
1	Crédito empresarial	Cupo rotativo	2944	129680131000	1
2	Crédito personas	Libranza	2943	132706871000	1
3	Crédito personas	Crédito de consumo	2944	130892514000	2
4	Depósito	Cuenta de ahorros	2944	132999415000	1
5	Depósito	CDT	2943	131741503000	2
6	Depósito	Cuenta corriente	2944	129863350000	3
7	Depósito empresarial	Cuenta empresarial	2943	131944823000	1
8	Fiducia	Fiducia inmobiliaria	2943	130222098000	1
9	Inversión	Fondo de inversión	2943	132867540000	1
10	Medios de pago	Tarjeta de crédito	2943	135233884000	1

```
WITH captacion AS (
    SELECT f.id_producto AS id_familia,
           f.nombre      AS familia,
           p.id_producto,
           p.nombre      AS producto,
           SUM(c.saldo)  AS saldo_captado,
           COUNT(*)      AS cuentas
      FROM cuenta c
      JOIN catalogo_producto p
        ON p.id_producto = c.id_producto
      JOIN catalogo_producto f
        ON f.id_producto = p.id_padre
     WHERE c.estado = 'ACTIVA'
     GROUP BY f.id_producto,
              f.nombre,
              p.id_producto,
              p.nombre
),
ranking AS (
    SELECT v.*,

           DENSE_RANK() OVER (
               PARTITION BY id_familia
               ORDER BY saldo_captado DESC
           ) AS puesto,

           ROUND(
               100 * RATIO_TO_REPORT(saldo_captado)
                     OVER (PARTITION BY id_familia),
               2
           ) AS porcentaje_familia

      FROM captacion v
)
SELECT familia,
       producto,
       cuentas,
       ROUND(saldo_captado) AS saldo_captad

.
```

Ejercicio 3.2 — Series de tiempo: acumulado y variación

PREGUNTA 3.2 — ¿Qué diferencia hay entre ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW y RANGE BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW? Constrúyase un caso donde den resultados distintos.

Respuesta: ROWS define la ventana utilizando la posición física o lógica de las filas dentro del orden establecido, mientras que RANGE trabaja con el valor de la expresión utilizada en el ORDER BY. Cuando existen valores repetidos en la columna de ordenamiento, RANGE considera conjuntamente todas las filas que tienen el mismo valor, mientras que ROWS las procesa individualmente. En la consulta mensual del taller ambos pueden producir el mismo resultado debido a que existe una sola fila por mes, pero su comportamiento cambia cuando existen empates en la columna utilizada para ordenar.

ESCRIBA 3.2 — Consulta: promedio móvil de los últimos 3 meses (pegar también en el .sql)

	MES	OPERACIONES	VALOR	ACUMULADO	MES_ANTERIOR	VAR_PCT
1	2026-01	5255	20971162000	20971162000	(null)	(null)
2	2026-02	4761	19194489000	40165651000	20971162000	-8,5
3	2026-03	5247	20878764000	61044415000	19194489000	8,8
4	2026-04	5084	20768947000	81813362000	20878764000	-0,5
5	2026-05	5246	21004496000	102817858000	20768947000	1,1
6	2026-06	5084	20347263000	123165121000	21004496000	-3,1
7	2026-07	5246	20816324000	143981445000	20347263000	2,3
8	2026-08	5276	21060883000	165042328000	20816324000	1,2
9	2026-09	5084	20443763000	185486091000	21060883000	-2,9
10	2026-10	5255	21371994000	206858085000	20443763000	4,5
11	2026-11	5093	20266310000	227124395000	21371994000	-5,2
12	2026-12	5265	21393061000	248517456000	20266310000	5,6

```
 WITH mensual AS (
    SELECT TRUNC(fecha_transf, 'MM') AS mes,
           SUM(valor)                AS valor_transferido,
           COUNT(*)                  AS operaciones
      FROM transferencia
     WHERE fecha_transf >= DATE '2026-01-01'
       AND fecha_transf <  DATE '2027-01-01'
       AND estado = 'APLICADA'
     GROUP BY TRUNC(fecha_transf, 'MM')
)
SELECT TO_CHAR(mes, 'YYYY-MM') AS mes,
       operaciones,
       ROUND(valor_transferido) AS valor,
```

```
        ROUND(
            SUM(valor_transferido) OVER (
                ORDER BY mes
                ROWS BETWEEN UNBOUNDED PRECEDING
                         AND CURRENT ROW
            )
        ) AS acumulado,

        ROUND(
            LAG(valor_transferido)
            OVER (ORDER BY mes)
        ) AS mes_anterior,

        ROUND(
            100 *
            (
                valor_transferido -
                LAG(valor_transferido) OVER (ORDER BY mes)
            )
            /
            NULLIF(
                LAG(valor_transferido) OVER (ORDER B
```

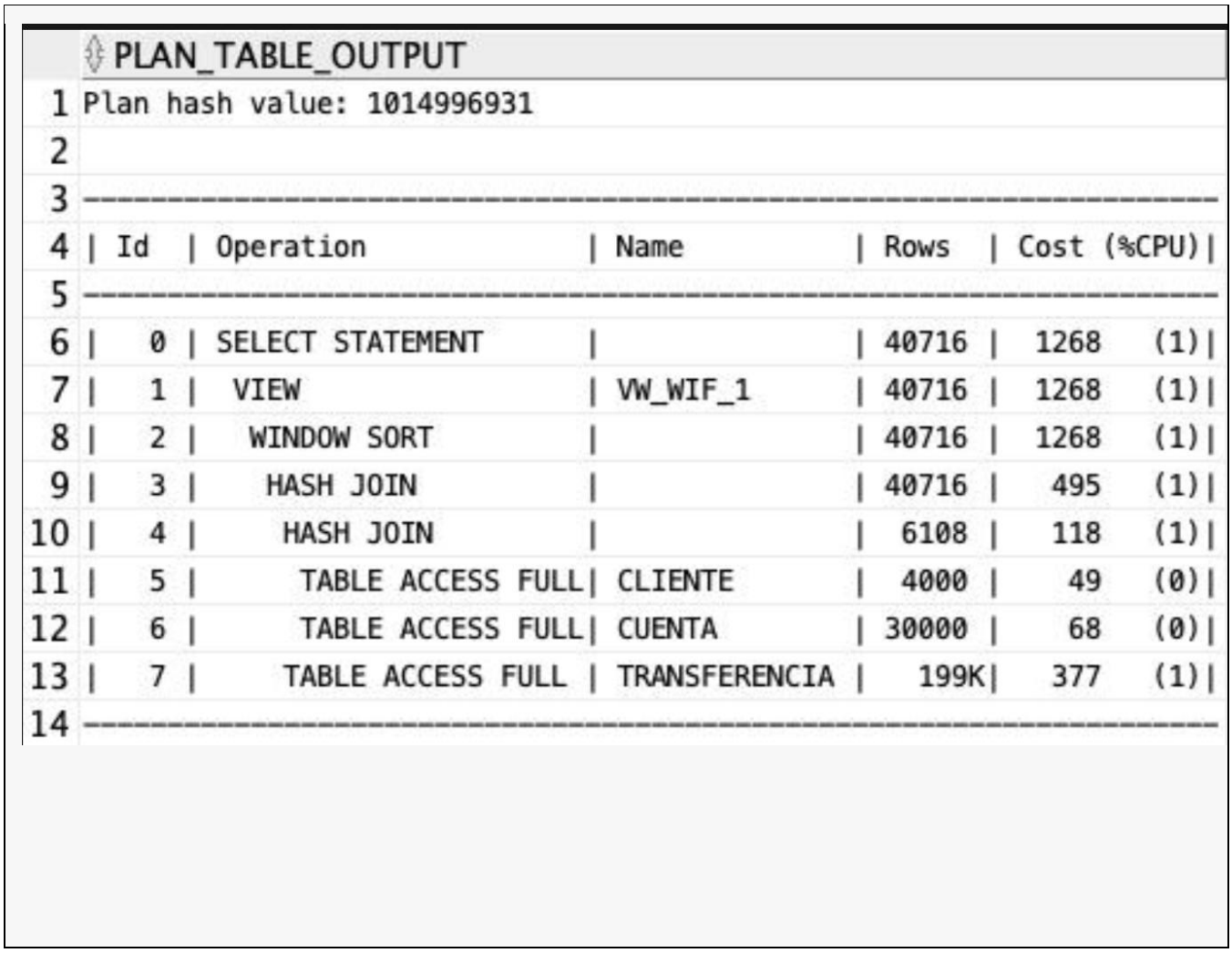
La parte nueva es:

```
AVG(valor_transferido) OVER (
  ORDER BY mes
  ROWS BETWEEN 2 PRECEDING AND CURRENT ROW
)
```

Es decir:

mes actual
+ mes anterior
+ dos meses atrás

Para enero tendrá solo enero; para febrero usará enero y febrero; desde marzo ya utilizará tres meses



```
PLAN_TABLE_OUTPUT
Plan hash value: 1014996931

-------------------------------------------------------------------------------
| Id  | Operation            | Name          | Rows  | Cost (%CPU)|
-------------------------------------------------------------------------------
|   0 | SELECT STATEMENT     |               | 40716 |  1268   (1)|
|   1 |  VIEW                | VW_WIF_1      | 40716 |  1268   (1)|
|   2 |   WINDOW SORT        |               | 40716 |  1268   (1)|
|   3 |    HASH JOIN         |               | 40716 |   495   (1)|
|   4 |     HASH JOIN        |               |  6108 |   118   (1)|
|   5 |      TABLE ACCESS FULL| CLIENTE       |  4000 |    49   (0)|
|   6 |      TABLE ACCESS FULL| CUENTA        | 30000 |    68   (0)|
|   7 |     TABLE ACCESS FULL | TRANSFERENCIA |   199K|   377   (1)|
-------------------------------------------------------------------------------
```

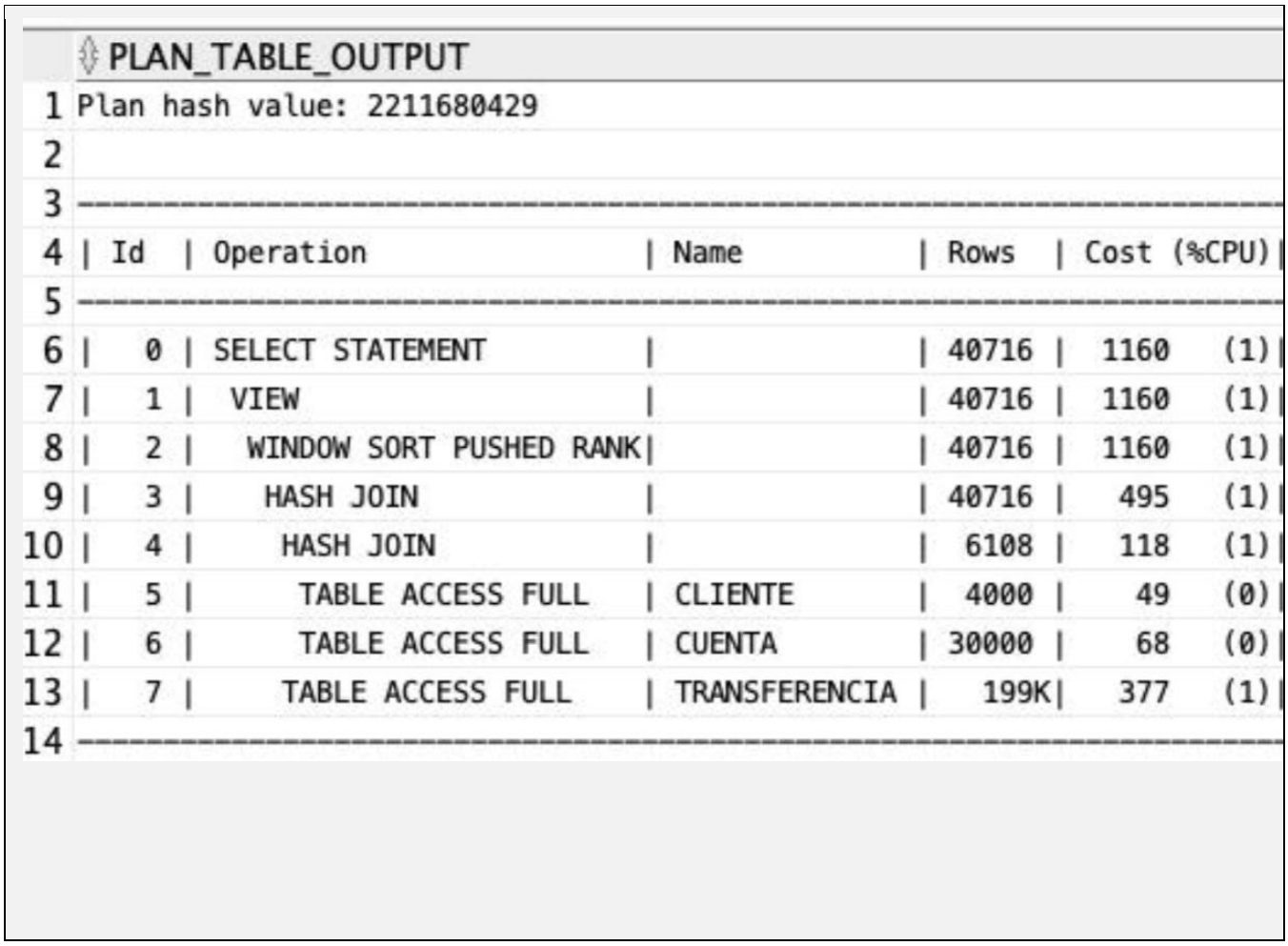
Ejercicio 3.3 — La misma respuesta, dos planes muy distintos

PREGUNTA 3.3 — Compare los dos planes (subconsulta correlacionada vs. función analítica). ¿Cuántas veces se lee TRANSFERENCIA en cada versión? ¿Cuál es el COST de cada una?

Respuesta: En la versión A, TRANSFERENCIA participa tanto en la consulta principal como en la subconsulta utilizada para obtener MAX(fecha_transf), por lo que el plan puede mostrar más de un acceso a la tabla. En la versión B, las transferencias se procesan en un único conjunto y posteriormente ROW_NUMBER() establece la posición de cada transferencia dentro de cada cliente. Por esta razón, la versión analítica evita la lógica repetitiva de la subconsulta correlacionada y generalmente presenta un plan más eficiente.

En la ejecución realizada, la versión A presentó un COST de 1268, mientras que la versión B presentó un COST de 1160. Por lo tanto, para este caso la versión B resulta más eficiente según la estimación del optimizador, ya que presenta un costo menor.

Aunque conceptualmente una subconsulta correlacionada podría implicar evaluaciones repetidas, el plan de ejecución obtenido demuestra que el optimizador de Oracle reescribió la consulta y evitó accesos redundantes a TRANSFERENCIA. Por esta razón, la comparación debe realizarse con base en el plan real obtenido y no únicamente en la estructura sintáctica de la consulta.



```
PLAN_TABLE_OUTPUT
Plan hash value: 2211680429

-----------------------------------------------------------------------------------
| Id  | Operation                | Name          | Rows  | Cost (%CPU)|
-----------------------------------------------------------------------------------
|   0 | SELECT STATEMENT         |               | 40716 |  1160   (1)|
|   1 |  VIEW                    |               | 40716 |  1160   (1)|
|   2 |   WINDOW SORT PUSHED RANK|               | 40716 |  1160   (1)|
|   3 |    HASH JOIN             |               | 40716 |   495   (1)|
|   4 |     HASH JOIN            |               |  6108 |   118   (1)|
|   5 |      TABLE ACCESS FULL   | CLIENTE       |  4000 |    49   (0)|
|   6 |      TABLE ACCESS FULL   | CUENTA        | 30000 |    68   (0)|
|   7 |     TABLE ACCESS FULL    | TRANSFERENCIA |   199K|   377   (1)|
-----------------------------------------------------------------------------------
```

PREGUNTA 3.4 — Las dos consultas NO son totalmente equivalentes: hay un caso en el que devuelven distinto número de filas. ¿Cuál es? (pista: piense en un cliente que hizo dos transferencias el mismo día a la misma hora, algo perfectamente normal en un pago de nómina)

Respuesta: Las consultas producen resultados diferentes cuando un mismo cliente tiene dos o más transferencias con exactamente la misma fecha y hora y estas corresponden a su fecha máxima. La versión basada en MAX(fecha_transf) devuelve todas las transferencias cuya fecha coincide con ese máximo. En cambio, ROW_NUMBER() asigna una única posición rn = 1, por lo que devuelve solamente una de ellas.

Ejercicio 3.4 — Jerarquías: la CTE recursiva

PREGUNTA 3.5 — ¿Qué ventaja tiene la CTE recursiva sobre CONNECT BY? ¿Y qué le da CONNECT BY que la CTE no?

Respuesta: La principal ventaja de una CTE recursiva es que expresa explícitamente el caso inicial y la regla de recursión, lo que facilita construir jerarquías junto con otras operaciones SQL y resulta conceptualmente más portable a otros gestores que soportan consultas recursivas. CONNECT BY, por su parte, es una sintaxis jerárquica propia de Oracle y suele ser más compacta para recorrer árboles. Además, Oracle proporciona con CONNECT BY herramientas específicas como LEVEL, PRIOR, SYS_CONNECT_BY_PATH, CONNECT_BY_ISLEAF, CONNECT_BY_ISCYCLE y ORDER SIBLINGS BY.

	JERARQUIA	NIVEL	RUTA
1	Banca Empresas	1	Banca Empresas
2	Crédito empresarial	2	Banca Empresas > Crédito empresarial
3	Cupo rotativo	3	Banca Empresas > Crédito empresarial > Cupo rotativo
4	Depósito empresarial	2	Banca Empresas > Depósito empresarial
5	Cuenta empresarial	3	Banca Empresas > Depósito empresarial > Cuenta empresarial
6	Banca Patrimonial	1	Banca Patrimonial
7	Fiducia	2	Banca Patrimonial > Fiducia
8	Fiducia inmobiliaria	3	Banca Patrimonial > Fiducia > Fiducia inmobiliaria
9	Inversión	2	Banca Patrimonial > Inversión
10	Fondo de inversión	3	Banca Patrimonial > Inversión > Fondo de inversión
11	Banca Personas	1	Banca Personas
12	Crédito personas	2	Banca Personas > Crédito personas
13	Crédito de consumo	3	Banca Personas > Crédito personas > Crédito de consumo
14	Libranza	3	Banca Personas > Crédito personas > Libranza
15	Depósito	2	Banca Personas > Depósito
16	CDT	3	Banca Personas > Depósito > CDT
17	Cuenta corriente	3	Banca Personas > Depósito > Cuenta corriente
18	Cuenta de ahorros	3	Banca Personas > Depósito > Cuenta de ahorros
19	Medios de pago	2	Banca Personas > Medios de pago
20	Tarjeta de crédito	3	Banca Personas > Medios de pago > Tarjeta de crédito

ESCRIBA 3.3 — Consulta: saldo total captado por línea de negocio, con CTE recursiva (pegar también en el .sql)

	ID_LINEA	LINEA_NEGOCIO	CUENTAS_ACTIVAS	SALDO_TOTAL_CAPTADO
1	1	Banca Personas	17661	793437537000
2	3	Banca Patrimonial	5886	263089638000
3	2	Banca Empresas	5887	261624954000

```
WITH arbol (
   id_producto,
   nombre,
   id_padre,
   id_linea,
   linea_negocio,
   nivel
) AS (

   -- Nivel raíz: línea de negocio
   SELECT id_producto,
          nombre,
```

```
           id_padre,
           id_producto AS id_linea,
           nombre      AS linea_negocio,
           1           AS nivel
      FROM catalogo_producto
     WHERE id_padre IS NULL

    UNION ALL

    -- Descendemos por el catálogo
    SELECT c.id_producto,
           c.nombre,
           c.id_padre,
           a.id_linea,
           a.linea_negocio,
           a.nivel + 1
      FROM catalogo_producto c
      JOIN arbol a
        ON c.id_padre = a.id_producto
)
SELECT a.id_linea,
       a.linea_negocio,
       COUNT(*)          AS cuentas_activas,
       ROUND(SUM(c.saldo)) AS saldo_total_captado
  FROM cuenta c
  JOIN arbol a
    ON a.id_producto = c.id_producto
 WHERE c.estado = 'ACTIVA'
   AND a.nivel = 3
 GROUP BY a.id_linea,
          a.linea_negocio
 ORDER BY saldo_total_captado DESC;
```

```
500 rows merged.

Elapsed: 00:00:00.473
```

Ejercicio 3.5 — MERGE: aplicar los ajustes del cierre mensual

PREGUNTA 3.6 — ¿Por qué se agrupa la tabla de ajustes dentro del USING en lugar de usarla directamente? ¿Qué error da Oracle si una misma cuenta aparece dos veces en el origen?

Respuesta: El origen del MERGE se agrupa por id_cuenta para garantizar que cada cuenta de la tabla destino tenga una única fila correspondiente en el origen. Si una misma cuenta aparece varias veces en AJUSTE_SALDO, Oracle tendría múltiples filas intentando actualizar la misma fila de CUENTA dentro del mismo MERGE, haciendo que la operación deje de ser determinística. En ese escenario Oracle puede generar el error ORA-30926, indicando que no puede obtener un conjunto estable de filas del origen. Al agrupar previamente los ajustes, todos los valores correspondientes a una cuenta se consolidan en un único delta.

PREGUNTA 3.7 — El GREATEST(..., 0) evita que el saldo quede negativo. En un banco eso NO es una solución: si una cuota de manejo excede el saldo, truncar a cero significa que el banco regaló la diferencia. ¿Qué haría usted en un sistema real? Proponga el mecanismo, no solo la crítica.

Respuesta: El problema está aquí:

GREATEST(c.saldo + a.delta, 0)

En un sistema bancario real el ajuste no debería truncarse mediante GREATEST. Antes de aplicar el movimiento se debería comprobar si existe saldo suficiente o si el producto permite sobregiro. Cuando el ajuste no pueda aplicarse completamente, la cuenta debe permanecer sin modificaciones y el ajuste debe cambiar a un estado como PENDIENTE, RECHAZADO o POR_GESTIONAR, registrando además el motivo y la fecha del intento. De esta manera se conserva la obligación completa y existe trazabilidad para que posteriormente pueda ser cobrada, compensada o procesada según las reglas del producto.

ESCRIBA 3.4 — MERGE que excluye las cuentas cuyo ajuste no se pueda aplicar completo, más la consulta de las excluidas (pegar también en el .sql)

```
 MERGE INTO cuenta c
USING (
    SELECT id_cuenta,
           SUM(delta) AS delta
      FROM ajuste_saldo
     GROUP BY id_cuenta
) a
ON (c.id_cuenta = a.id_cuenta)

WHEN MATCHED THEN
UPDATE SET
    c.saldo   = c.saldo + a.delta,
    c.version = c.version + 1
```

```
WHERE c.saldo + a.delta >= 0;
```

Ejercicio 3.6 — PIVOT y LISTAGG

PREGUNTA 3.8 — ¿Qué diferencia hay entre FETCH FIRST 10 ROWS ONLY y FETCH FIRST 10 ROWS WITH TIES? ¿En cuál de los dos casos puede devolver más de 10 filas y por qué?

Respuesta: FETCH FIRST 10 ROWS ONLY limita el resultado a un máximo de diez filas, independientemente de que existan empates en el valor utilizado en el ORDER BY. En cambio, FETCH FIRST 10 ROWS WITH TIES devuelve las primeras diez posiciones y además incluye todas las filas adicionales que estén empatadas con la última fila incluida según el criterio de ordenamiento. Por esta razón, WITH TIES puede devolver más de diez registros.